

CARRERA	NOMBRE DE LA ASIGNATURA
INGENIERÍA EN SISTEMAS COMPUTACIONALES	LENGUAJES Y AUTÓMATAS II

DOCENTE DESIGNADO
ISC. RICARDO GONZÁLEZ GONZÁLEZ

EQUIPO / INTEGRANTES
HERNANDEZ PERES JUAN JOSE. RAMIREZ VAZQUEZ DIEGO. CAÑADA GARCIA ALI VLADIMIR.

NOMBRE Y DESCRIPCIÓN
CSV – QUERY Es un lenguaje de consultas, diseñado para poder manipular archivos CSV mediante operaciones, filtros y consultas avanzadas. El sistema implementara un pequeño compilador compuesto por análisis léxico, sintáctico y semántico, además de generar código Python con ayuda de Pandas para poder ejecutar instrucciones sobre los datos.

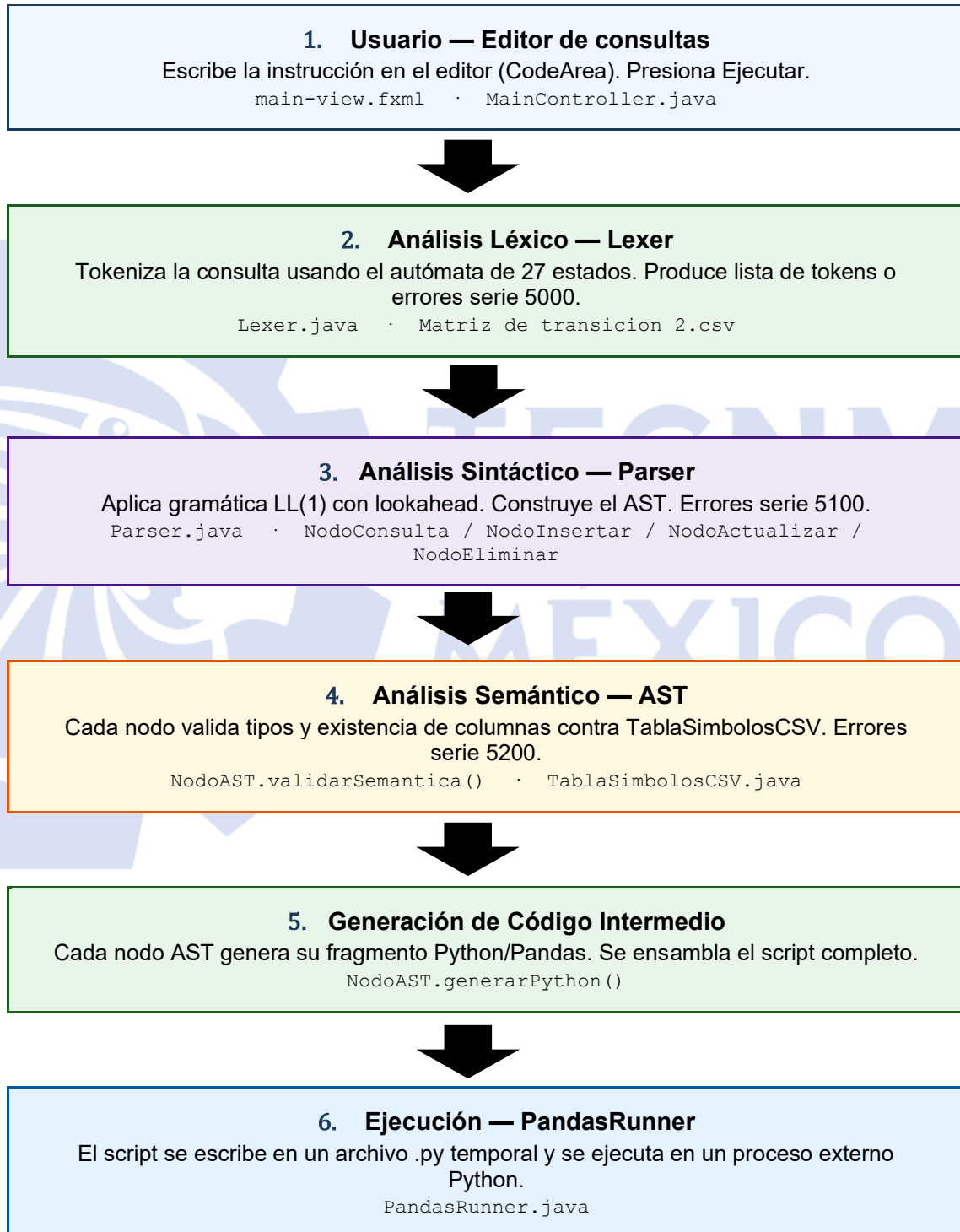
DURACIÓN
80 horas

MATERIALES UTILIZADOS
- Java JDK 17, es en lenguaje principal que vamos a utilizar para el desarrollo del compilador y la lógica que debe de llevar nuestro sistema. - IntelliJ IDEA, entorno que estaremos utilizando para desarrollar, programar y gestionar nuestro proyecto. - JavaFX, es el framework que utilizaremos para el desarrollo de la interfaz gráfica. - OpenCSV, es la librería la cual nos ayudara para la lectura y manejo de los archivos CSV. - Python3, lenguaje utilizado para ejecutar automáticamente las consultas que se genera. - Pandas, es la librería de Python que nos ayudara para la manipulación y el proceso de los datos tabulares. - Git, herramientas que utilizamos para el control de versiones del proyecto. - GitHub, es la plataforma que utilizamos para el almacenamiento y respaldo del repositorio. - Archivos CSV, los utilizamos como base de datos para poder almacenar la información que podremos manipular por el lenguaje. - Autómatas, los utilizamos para el análisis léxico y reconocimiento de los tokens. - AST (Arbol sintactico abstracto), es una estructura que nos ayudara a poder representar las consultas de manera interna del lenguaje.

1	INTRODUCCIÓN
	En la actualidad, muchos de los archivos CSV son muy utilizados para poder almacenar información de manera tabular debido a la simplicidad, portabilidad y compatibilidad que tiene con muchos sistemas y aplicaciones. Sin embargo, la manipulación manual de estos archivos nos puede resultar un poco complejo cuando requerimos hacer búsquedas, filtros, modificaciones o incluso eliminación de datos de manera eficiente. Con el objetivo de poder facilitar este proceso, se desarrolló CSV- Query, CSV Query Language es un lenguaje de dominio específico (DSL) diseñado para realizar operaciones de consulta y manipulación sobre archivos CSV. El lenguaje permite al usuario expresar instrucciones en español estructurado que son procesadas por un compilador de tres fases — léxico, sintáctico y semántico — para posteriormente generar y ejecutar código Python usando la biblioteca Pandas. El proyecto fue implementado en Java con JavaFX para la interfaz gráfica. El compilador sigue el pipeline clásico: análisis léxico mediante un autómata finito determinista cargado desde una matriz de transición, análisis sintáctico descendente recursivo que construye un Árbol de Sintaxis Abstracta (AST), y análisis semántico distribuido en cada nodo del AST. CSV-Query no solo nos permite gestionar archivos CSV de forma sencilla, sino que también nos ayuda como una aplicación de practica de conceptos fundamentales de compiladores, lenguajes formales, estructuras de datos y procesamiento de información tabular.
3	OBJETIVO GENERAL
	Diseñar e implementar un lenguaje de consultas que este orientado a la modificación de archivos CSV, basado en algunas técnicas de compiladores y lenguajes formales, que sea capaz de realizar operaciones CRUD (Create o insertar, Read o seleccionar, Update o actualizar, y finalmente Delete o eliminar) filtros y algunas consultas avanzadas mediante alguna sintaxis que esta inspirada en SQL solamente cambiando las palabras reservadas a nuestro lenguaje que es español.
4	OBJETIVOS ESPECÍFICOS
	- Implementar un analizador léxico utilizando autómatas finitos para el reconocimiento de tokens. - Desarrollar un parser descendente recursivo para interpretar la gramática del lenguaje. - Construir un Árbol Sintáctico Abstracto para así poder representar internamente las consultas. - Implementar validaciones semánticas para poder detectar errores en nombres de columnas y tipos de datos. - Generar automáticamente código en Python utilizando Python para así ejecutar las consultas sobre el archivo CSV. - Implementar operaciones CRUD (Insertar, Leer, Actualizar y Eliminar) sobre el archivo CSV que se este trabajando. - Diseñar una interfaz gráfica utilizando JavaFX para facilitar la interacción con el usuario. - Facilitar la manipulación de datos tabulares mediante un lenguaje sencillo y similar a SQL.

5	MARCO TEÓRICO
	Las consultas o queries son instrucciones las cuales nos ayudan a recuperar, modificar o incluso manipular información almacenada en fuente de datos, Por lo regular este tipo de operaciones se realizan sobre bases de datos mediante lenguajes como SQL, sin embargo, los archivos CSV también pueden ser tratados como estructuras tabulares, algo muy similar a una tabla de base de datos. Un archivo CSV nos almacena información que puede estar organizado en filas y columnas, donde cada línea representa un registro y cada valor está separado por comas. Debido a esta estructura tabular, es posible poder aplicar algunas operaciones de consulta y manipulación similares a las utilizadas en sistemas gestores de bases de datos. Los lenguajes formales son sistemas de símbolos y algunas reglas utilizadas para poder definir estructuras que son válidas dentro de un lenguaje de programación o en este caso de consulta. Estos lenguajes nos permiten establecer una gramática muy específica la cual va a determinar como deben de escribirse las instrucciones para que estas puedan ser interpretadas correctamente por un compilador o interprete. En este proyecto CSV – Query diseñamos una sintaxis que esta mas que nada basada en SQL para así poder realizar las consultas y manipular los archivos CSV mediante reglas gramaticales definidas. Un compilador es un programa el cual es encargado de traducir instrucciones escritas en un lenguaje fuente o representación ejecutable. Este proceso de compilación generalmente lo vamos a dividir en distintas etapas, las cuales son análisis léxico, análisis sintáctico, análisis semántico y al final la generación de código intermedio, este proceso es muy importante para que el usuario a partir de lo que digite pueda ir respetando las reglas y normas que el lenguaje tiene. Nuestro proyecto se le implementaran estas etapas para poder interpretar las consultas que el usuario ingrese y poder convertirlas en código Python ejecutable. Hablando de los distintos tipos de análisis, el análisis léxico es la primera etapa del compilador, el cual su función principal es poder leer el código fuente carácter por carácter para así poderlo agrupar en unidades llamadas tokens. Estos tokens lo que nos van a representar en el lenguaje pueden ser como las palabras reservadas, los identificadores, operadores, números y algunas cadenas de texto. En este caso se realizo el uso de un autómata finito y una tabla de transiciones, un autómata finito es un modelo matemático que se utiliza para poder reconocer patrones dentro de una cadena de caracteres, y este compuesto por: estados, transiciones, estado inicial, estados de aceptación. Después del análisis léxico, se utiliza el análisis sintáctico el cual se va a encargar de verificar que la secuencia de tokens generada por el leer nos cumpla con la gramática que tiene definida el leguaje, también se puede implementar mediante parser descendente recursivo que sea capaz de interpretar ya sea consultas select, como operaciones insert, update, Delete y de igual forma condiciones lógicas y expresiones aritméticas. El análisis semántico se va a encargar de verificar que las instrucciones tengan coherencia lógica dentro del contexto del programa, ya que no se podrán poner tokens o lexemas en cualquier orden si no que deben de llevar una secuencia. Todo esto lo haremos a través de Java, pero inicialmente lo que nos va a ingresar a la información del archivo CSV será Python y pandas, Python es un lenguaje de programación ampliamente utilizado en ciencia de datos y automatización. Pandas es una librería especializada en la manipulación de datos tabulares mediante estructuras llamadas DataFrames. En el proyecto, el compilador va a generar automáticamente scripts en Python utilizando pandas para poder ejecutar las consultas realizadas sobre los archivos CSV.

El sistema sigue un pipeline lineal de compilación. Cada etapa consume la salida de la anterior. Si cualquier etapa detecta un error, el proceso se detiene y la consola de compilación muestra el error con su código identificador.





7. Resultado — Interfaz

La salida CSV del proceso se parsea y se muestra en la TableView de la pestaña Resultado.

```
MainController.mostrarResultadoCSV()
```

Análisis Léxico

El análisis léxico es la primera fase del compilador. Su responsabilidad es leer la consulta carácter por carácter y agruparlos en unidades con significado llamadas tokens. Lo que distingue a este Lexer es que la lógica de transición no está codificada directamente en Java, sino que se carga dinámicamente desde un archivo CSV que representa la matriz de transición del autómata finito determinista.

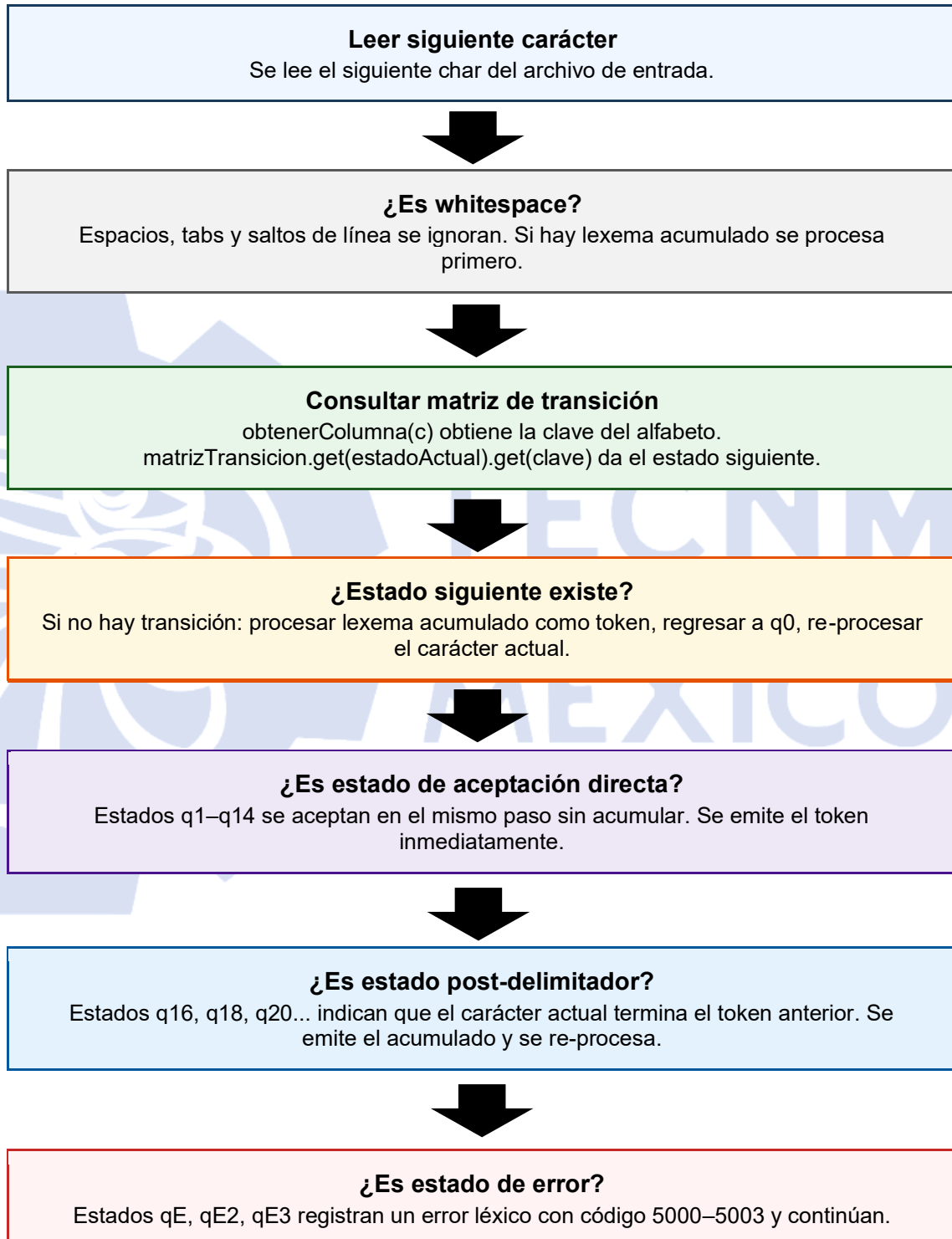
Carga del Autómata

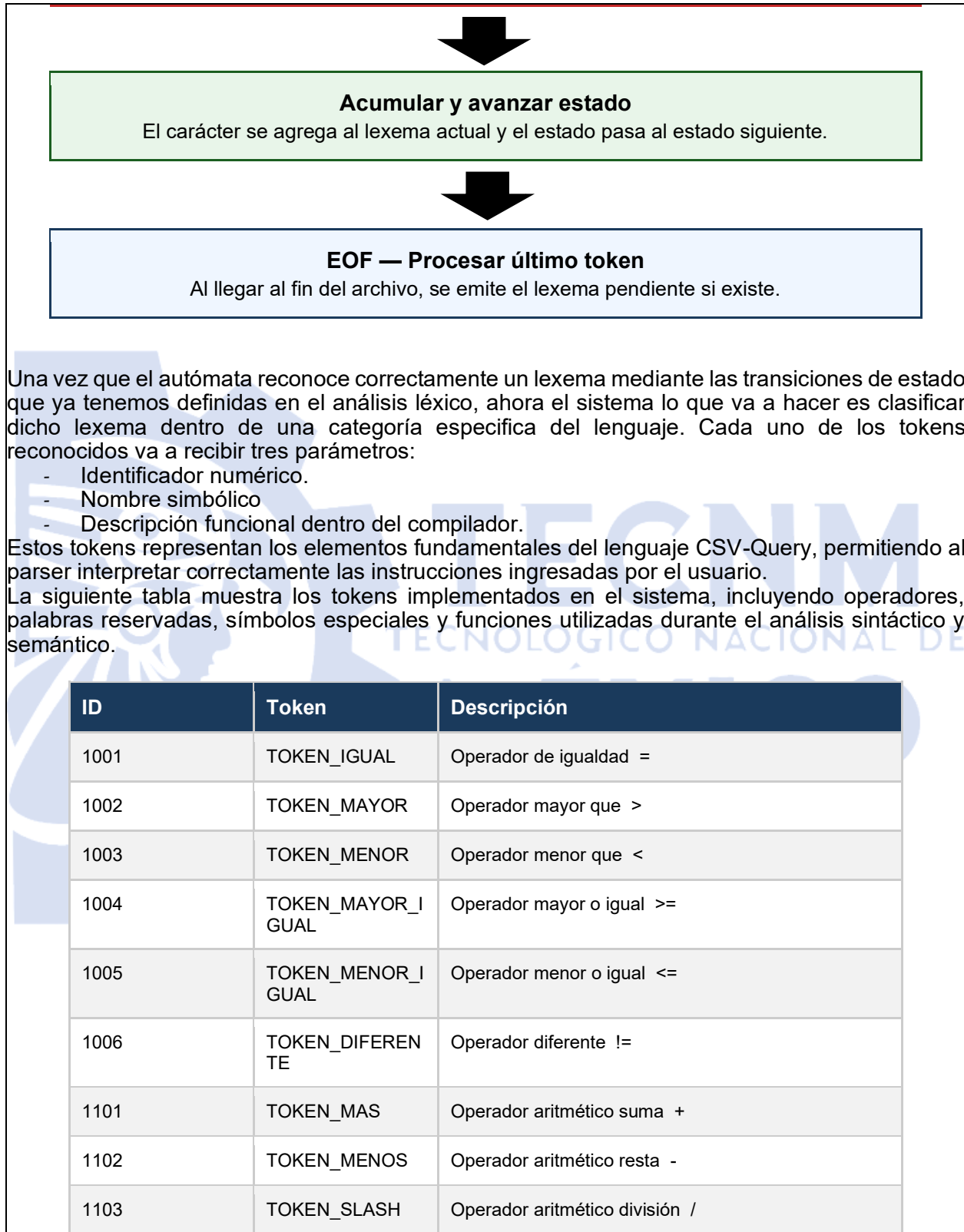
Al iniciar el Lexer se invoca el método cargarAutómata() que lee el archivo Matriz de transicion 2.csv usando la biblioteca OpenCSV. Cada fila representa un estado (q0, q1, ..., q26) y cada columna representa un símbolo del alfabeto. La celda [estado][símbolo] contiene el estado destino.

Durante la carga, los encabezados de columna son sanitizados con el método sanitizarEncabezado() para eliminar el apóstrofo que Excel/LibreOffice inyectan al símbolo '=' al guardar el archivo CSV (lo convierte en '=' que hace que TOKEN_IGUAL nunca se reconozca). Esta corrección es transparente al usuario.

```
// Sanitizar encabezados corruptos por Excel:
// '=' se guarda como "='" - el apóstrofo se elimina aquí
private String sanitizarEncabezado(String encabezado) {
    String s = encabezado.trim();
    if (s.length() > 1 && s.endsWith("'")) s = s.substring(0, s.length()-1);
    if (s.length() > 1 && s.startsWith("'")) s = s.substring(1);
    return s;
}
```

Diagrama de Flujo — Análisis Léxico





2001	TOKEN_TRAER	Palabra reservada TRAER (SELECT)
2002	TOKEN_DESDE	Palabra reservada DESDE (FROM)
2003	TOKEN_DONDE	Palabra reservada DONDE (WHERE)
2004	TOKEN_ORDENAR	Palabra reservada ORDENAR (ORDER BY)
2005	TOKEN_POR	Palabra reservada POR
2006	TOKEN_Y	Operador lógico Y (AND)
2007	TOKEN_O	Operador lógico O (OR)
2008	TOKEN_LIMITAR	Palabra reservada LIMITAR (LIMIT)
2009	TOKEN_DISTINTO	Palabra reservada DISTINTO (DISTINCT)
2010	TOKEN_ASC	Modificador de orden ascendente
2011	TOKEN_DESC	Modificador de orden descendente
2013	TOKEN_CONTAR	Función de agregación CONTAR (COUNT)
2014	TOKEN_PROMEDIO	Función de agregación PROMEDIO (AVG)
2015	TOKEN_SUMA	Función de agregación SUMA (SUM)
2016	TOKEN_MAXIMO	Función de agregación MAXIMO (MAX)
2017	TOKEN_MINIMO	Función de agregación MINIMO (MIN)
2018	TOKEN_TODO	Comodín de todas las columnas (*)
2019	TOKEN_GUARDAR	Palabra reservada GUARDAR
2020	TOKEN_EN	Palabra reservada EN
2021	TOKEN_INSERTAR	Palabra reservada CRUD INSERTAR (INSERT)
2022	TOKEN_VALORES	Palabra reservada CRUD VALORES (VALUES)
2023	TOKEN_ACTUALIZAR	Palabra reservada CRUD ACTUALIZAR (UPDATE)

2024	TOKEN_ELIMINAR	Palabra reservada CRUD ELIMINAR (DELETE)
3001	TOKEN_COMA	Símbolo separador ,
3002	TOKEN_PUNTOCOMA	Símbolo fin de consulta ;
3003	TOKEN_ASTERISCO	Símbolo asterisco / multiplicación *
3004	TOKEN_PARENTESIS_ABRE	Símbolo paréntesis abre (
3005	TOKEN_PARENTESIS_CIERRA	Símbolo paréntesis cierra)
4001	TOKEN_ID	Identificador de columna o nombre
4002	TOKEN_ENTERO	Literal numérico entero (ej: 30)
4003	TOKEN_FLOAT	Literal numérico decimal (ej: 0.5)
4004	TOKEN_STRING	Literal de cadena entre comillas (ej: "Madrid")

Códigos de Error Léxico

Además del reconocimiento de tokens que ya son válidos, el analizador léxico implementa un sistema de detección y reporte de errores durante el recorrido del autómata. Cuando una secuencia de caracteres no cumple con las transiciones definidas en la matriz léxica, el sistema genera un código en donde se va a especificar el error para así facilitar la identificación del problema. Estos errores nos van a permitir detectar situaciones como caracteres inválidos, operadores incompletos, cadenas sin cerrar, entre otros errores para validar antes de pasar a los siguientes analizadores. La siguiente tabla muestra los principales códigos de error léxico:

Código	Serie	Descripción
5000	Léxico — genérico	Carácter no reconocido por ninguna transición del autómata
5001	Léxico — operador	Operador incompleto: el carácter ! sin = a continuación
5002	Léxico — cadena	Cadena de texto iniciada con " pero sin cierre antes del fin de línea
5003	Léxico — número	Número con formato inválido (ej: 18.3.2 con dos puntos decimales)

Análisis Sintáctico

El análisis sintáctico toma la lista de tokens producida por el Lexer y verifica que su orden y combinación respete la gramática del lenguaje. Si la estructura es válida construye un Árbol de Sintaxis Abstracta (AST). El parser es de tipo descendente recursivo (LL) con lookahead de un token para resolver ambigüedades.

Gramática del Lenguaje

Para que el parser se pueda ejecutar de manera correcta las instrucciones del lenguaje que le estamos dando, es necesario definir una gramática la cual va a establecer una estructura válida de cada consulta. Esta gramática describe las reglas sintácticas que son utilizadas por el compilador para así reconocer operaciones como consultas, inserciones, actualizaciones, eliminaciones, expresiones aritméticas y condiciones lógicas.

```
operacion    → consulta | insertar | actualizar | eliminar

consulta     → TRAER [DISTINTO] seleccion DESDE STRING
               [DONDE condicion] [ORDENAR POR ID (ASC|DESC)]
               [LIMITAR ENTERO] [GUARDAR EN STRING] ;

seleccion    → * | TODO | agregacion | col {, col}*
agregacion   → (CONTAR|SUMA|PROMEDIO|MAXIMO|MINIMO) ( * | ID )

insertar     → INSERTAR EN STRING ( col {, col}* )
               VALORES ( val {, val}* ) ;

actualizar   → ACTUALIZAR STRING VALORES col=val {, col=val}*
               DONDE condicion ;

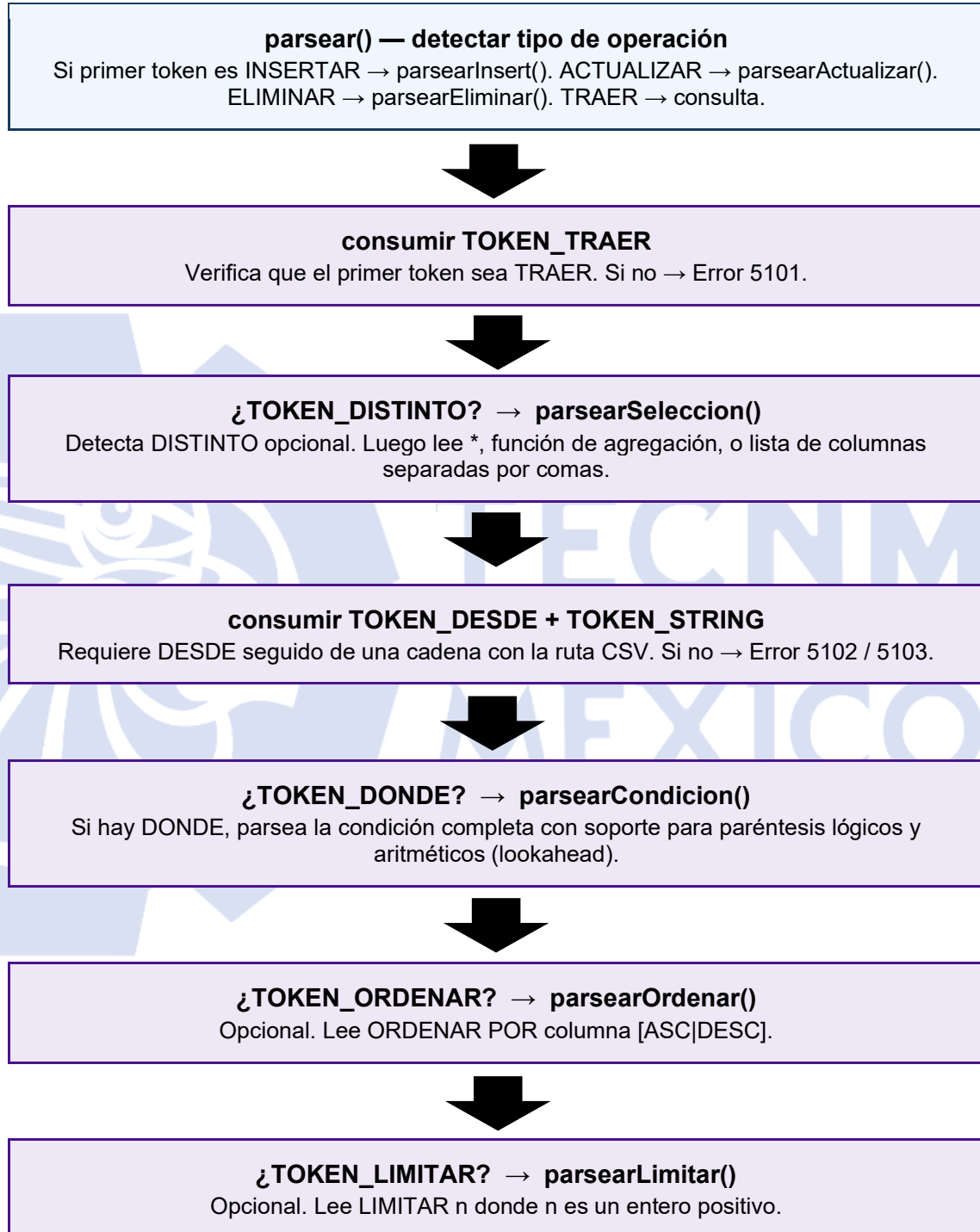
eliminar     → ELIMINAR STRING DONDE condicion ;

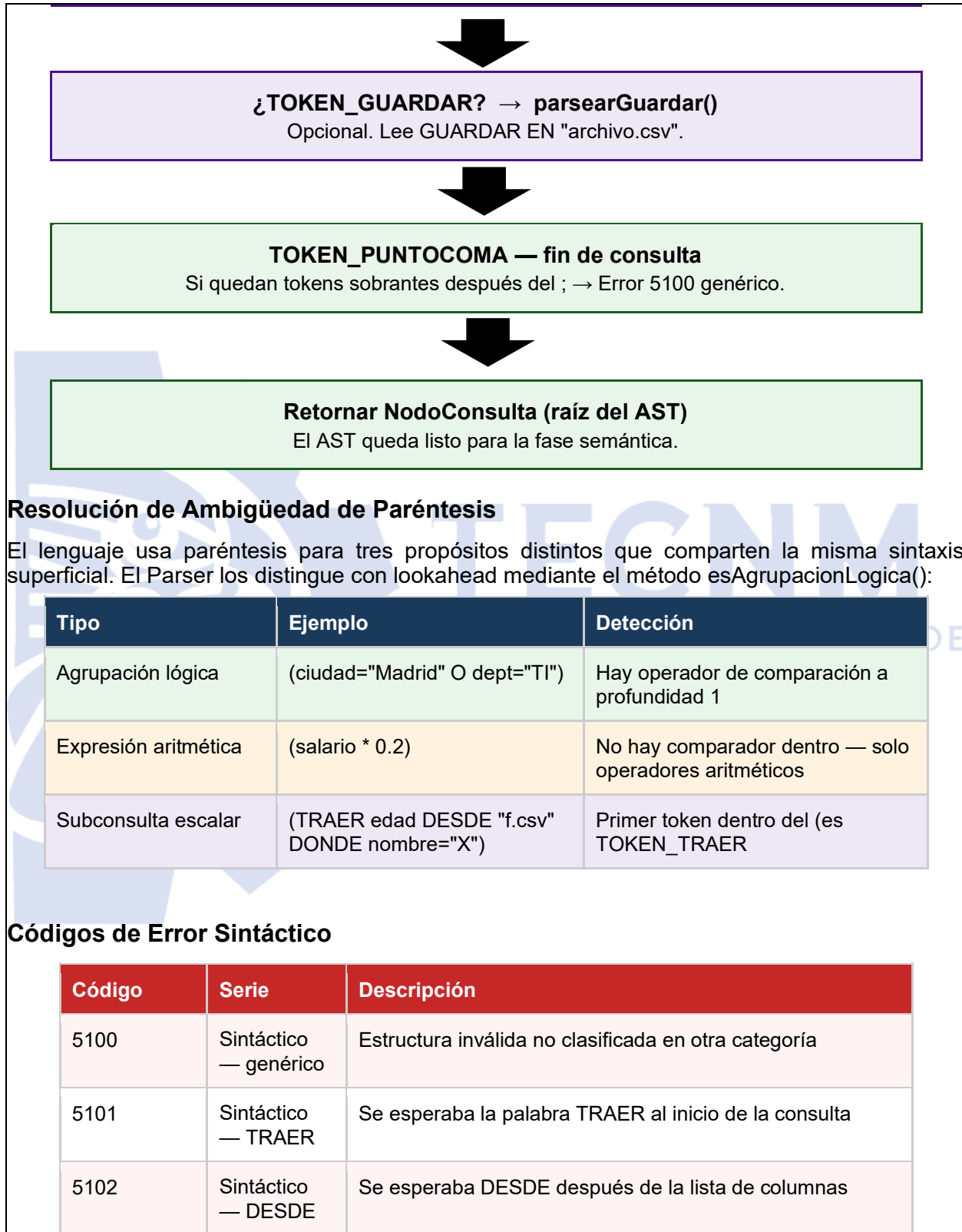
condicion     → termino { (Y|O) termino }*
termino      → ( condicion )           -- agrupación lógica
               | comparacion

comparacion   → expresion OPERADOR expresion
expresion     → ( TRAER ... )           -- subconsulta escalar
               | ( expr_arit )          -- aritmética agrupada
               | col [op_arit col|lit]   -- col o col OPERADOR lit
               | literal

op_arit       → * | + | - | /
OPERADOR      → = | > | < | >= | <= | !=
```

Diagrama de Flujo — Análisis Sintáctico (TRAER)





5103	Sintáctico — CSV	Se esperaba una ruta de archivo CSV entre comillas
5104	Sintáctico — operador	Se esperaba un operador de comparación (=, >, <, >=, <=, !=)
5105	Sintáctico — paréntesis	Paréntesis sin cerrar en la condición
5106	Sintáctico — punto y coma	Se esperaba ; al final de la consulta
5107	Sintáctico — columna	Nombre de columna inválido o inesperado
5108	Sintáctico — literal	Se esperaba un valor literal (número o cadena de texto)

Análisis Semántico

El análisis semántico verifica que la consulta tenga sentido más allá de su estructura gramatical. Mientras el análisis sintáctico valida la forma, el semántico valida el significado: ¿existe la columna que se menciona? ¿Son compatibles los tipos en una comparación? ¿Se puede aplicar este operador a estos datos?

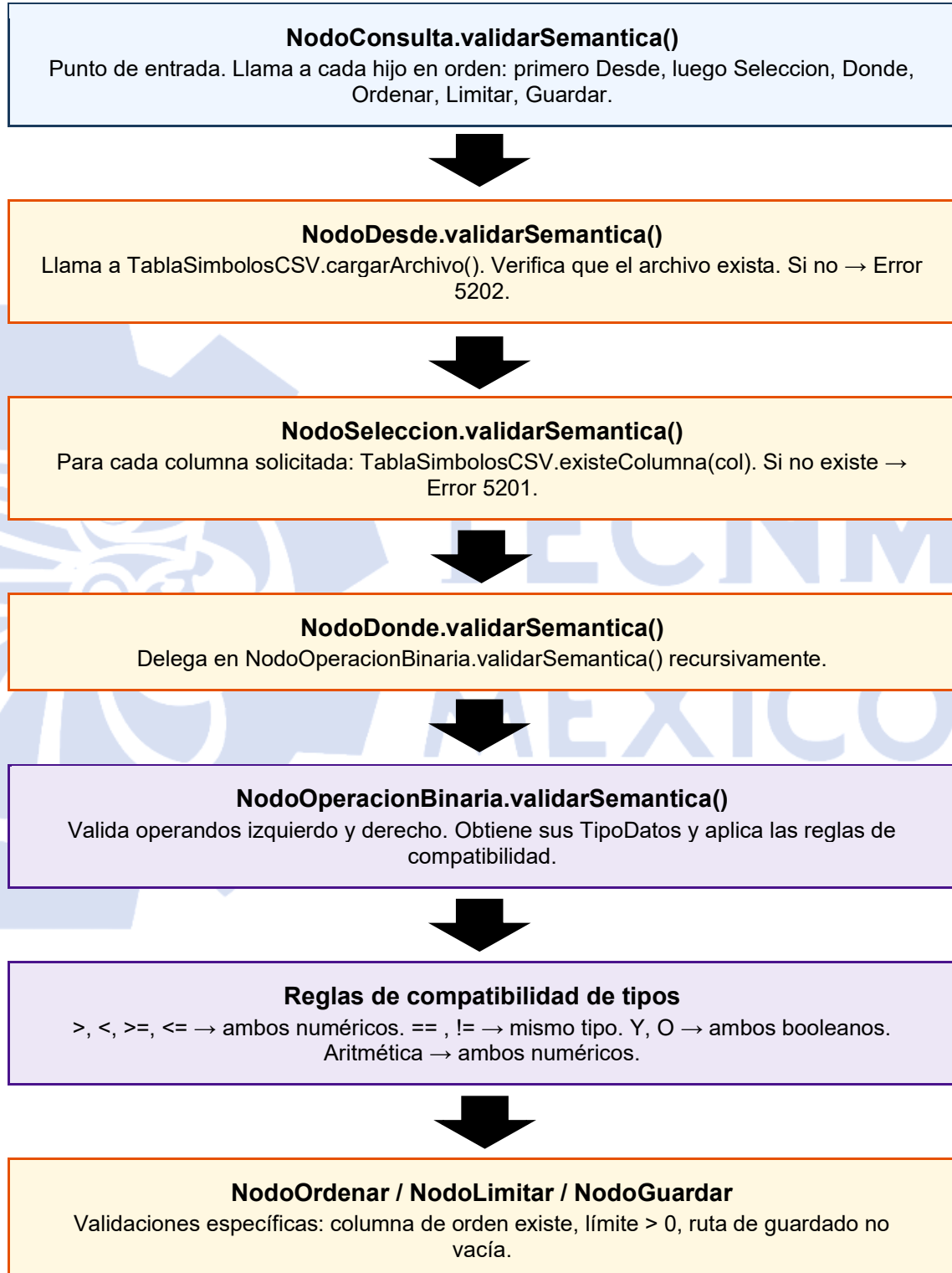
En este compilador el análisis semántico está distribuido: cada nodo del AST implementa el método validarSemantica() y es responsable de validar su propia lógica. La validación comienza en NodoConsulta (la raíz) y se propaga hacia los hijos en el orden correcto.

TablaSimbolosCSV

TablaSimbolosCSV es el componente central del análisis semántico. Al validar NodoDesde, se llama a TablaSimbolosCSV.cargarArchivo(ruta) que lee el encabezado del CSV y la primera fila de datos para inferir el tipo de cada columna.

Método	Descripción
cargarArchivo(ruta)	Lee la primera fila (encabezado) e infiere tipos analizando la segunda fila de datos
existeColumna(nombre)	Retorna true si la columna existe en el mapa
obtenerTipoColumna(nombre)	Retorna TipoDato.ENTERO / DECIMAL / CADENA / BOOLEANO o ERROR si no existe
inferirTipo(valor)	Lógica de inferencia: solo dígitos → ENTERO, dígitos con punto → DECIMAL, true/false → BOOLEANO, resto → CADENA

Diagrama de Flujo — Análisis Semántico




Validación exitosa → generarPython()

Solo si todas las validaciones pasan se procede a la generación de código.

Nodos del AST

Cada nodo implementa la interfaz `NodoAST` con tres métodos: `validarSemantica()`, `generarPython()` y `toInfo()` (para la visualización del árbol en la UI).

Nodo	Semántica	Python generado
NodoConsulta	Orquesta la validación de todos los hijos en orden	import pandas as pd + ensamble completo del script
NodoSeleccion	Verifica existencia de cada columna en TablaSimbolosCSV	df = df[['col1','col2']] o df = df[*]
NodoDesde	Carga TablaSimbolosCSV desde el archivo indicado	df = pd.read_csv(r"ruta")
NodoDonde	Delega en la expresión lógica interna	df = df[condicion]
NodoOperacionBinaria	Compatibilidad de tipos entre operandos	(df["col"] op valor) o (exp1 & exp2)
NodoExpresionAritmetica	Ambos operandos numéricos	(df["col"] * 0.2)
NodoSubconsulta	Valida la consulta anidada completa	bloque _sub0_df + _sub0 = ...iloc[0]
NodoAgregacion	Columna existe y es de tipo compatible	print(df["col"].mean()) etc.
NodoOrdenar	Columna de orden existe	df = df.sort_values(by='col', ascending=T/F)
NodoLimitar	n > 0	df = df.head(n)
NodoGuardar	Ruta no vacía	df.to_csv(r"ruta", index=False)
NodoLiteral	Sin semántica propia — retorna su TipoDato	Valor directo: 30, 'Madrid', 0.5
NodoColumna	Sin semántica propia — el padre valida	Nombre de columna entre comillas

NodoInsertar	Columnas existen, cantidad col==val	df.loc[len(df)] = nueva_fila
NodoActualizar	Columnas existen, condición válida	df.loc[condicion, 'col'] = valor
NodoEliminar	Archivo existe, condición válida	df = df.loc[~condicion]

Códigos de Error Semántico

Código	Serie	Descripción
5200	Semántico — genérico	Error semántico no clasificado en otra categoría
5201	Semántico — columna	La columna especificada no existe en el archivo CSV
5202	Semántico — archivo	El archivo CSV especificado no se encontró en el sistema
5203	Semántico — operador orden	Operadores >, <, >= y <= no pueden aplicarse a columnas de tipo texto
5204	Semántico — tipos	Tipos incompatibles en una comparación (ej: número comparado con cadena)
5205	Semántico — aritmética	Operador aritmético aplicado a una columna que no es numérica
5206	Semántico — lógico	Operadores Y/O aplicados a expresiones que no devuelven booleano
5207	Semántico — agregación	Error en función de agregación (columna no numérica para PROMEDIO/SUMA)
5210	Ejecución Python	Error en tiempo de ejecución del script Python generado (Pandas)

Generación de Código Intermedio

Una vez que el análisis semántico concluye sin errores, se recorre el AST una segunda vez llamando al método generarPython() de cada nodo. El resultado es un script Python completo que usa la biblioteca Pandas para ejecutar la operación. Este script es visible para el usuario en la pestaña Código Intermedio de la interfaz.

Para la instalación de pandas vamos a descargar esta librería directamente en nuestro proyecto, en la terminal de nuestro proyecto.

```
PS C:\Users\W10\IdeaProjects\CSV-Query> python --version
Python 3.14.3

PS C:\Users\W10\IdeaProjects\CSV-Query> pip install pandas
Defaulting to user installation because normal site-packages is not writeable
Collecting pandas
  Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl.metadata (19 kB)
Collecting numpy>=2.3.3 (from pandas)
  Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl.metadata (6.6 kB)
Collecting python-dateutil<=2.9.2 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Downloading tzdata-2026.1-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting six>=1.5 (from python-dateutil<=2.9.2-pandas)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Downloading pandas-3.0.2-cp314-cp314-win_amd64.whl (9.9 MB)
9.9/9.9 MB 2.3 MB/s 0:00:04
Downloading numpy-2.4.4-cp314-cp314-win_amd64.whl (12.4 MB)
12.4/12.4 MB 2.3 MB/s 0:00:05
Using cached python_dateutil-2.9.0.post0-py3-none-any.whl (229 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Downloading tzdata-2026.1-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, six, numpy, python-dateutil, pandas
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed numpy-2.4.4 pandas-3.0.2 python-dateutil-2.9.0.post0 six-1.17.0 tzdata-2026.1

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

PS C:\Users\W10\IdeaProjects\CSV-Query>
```

Cada nodo genera de forma independiente su fragmento de código. NodoConsulta ensambla los fragmentos en el orden correcto:

importaciones → lectura CSV → filtros → ordenamiento → límite → proyección → salida.

Ejemplo — Consulta de selección

```
-- Consulta en el lenguaje CSV Query:
TRAER nombre, salario DESDE "datos.csv" DONDE salario > 50000
ORDENAR POR salario DESC LIMITAR 5;

-- Código Python generado:
import pandas as pd
import sys

df = pd.read_csv(r"datos.csv")
df = df[(df["salario"] > 50000)]
df = df[["nombre", "salario"]]
df = df.sort_values(by="salario", ascending=False)
df = df.head(5)
print(df.to_csv(index=False))
```

Ejemplo — Subconsulta escalar

```
-- Consulta con subconsulta escalar:
TRAER nombre, edad DESDE "datos.csv"
DONDE edad > (TRAER edad DESDE "datos.csv" DONDE nombre = "Carlos
Mendoza");

-- Código Python generado (bloque auxiliar primero):
_sub0_df = pd.read_csv(r"datos.csv")
_sub0_df = _sub0_df[_sub0_df["nombre"] == "Carlos Mendoza"]
_sub0 = _sub0_df["edad"].iloc[0]
df = pd.read_csv(r"datos.csv")
df = df[(df["edad"] > _sub0)]
df = df[["nombre", "edad"]]
print(df.to_csv(index=False))
```

Ejemplo — Expresión aritmética en condición

```
-- Filtrar por bono estimado (20% del salario):
TRAER * DESDE "datos.csv" DONDE (salario * 0.2) < 10000;

-- Código Python generado:
df = pd.read_csv(r"datos.csv")
df = df[((df["salario"] * 0.2) < 10000)]
print(df.to_csv(index=False))
```

Operaciones CRUD

Además de las consultas de lectura, el lenguaje soporta las tres operaciones de modificación del CRUD: INSERTAR, ACTUALIZAR y ELIMINAR. Estas operaciones modifican el archivo CSV directamente y muestran el estado actualizado del archivo en la tabla de resultados.

INSERTAR — Agregar un nuevo registro

```
-- Sintaxis:
INSERTAR EN "archivo.csv" (col1, col2, col3) VALORES (val1, val2, val3);

-- Ejemplo:
INSERTAR EN "datos.csv" (id, nombre, edad, salario)
VALORES (031, "Ana García", 27, 52000);

-- Python generado:
df = pd.read_csv("datos.csv")
nueva_fila = {"id": 031, "nombre": "Ana García", "edad": 27, "salario": 52000}
df.loc[len(df)] = nueva_fila
df.to_csv("datos.csv", index=False)
print(df.to_csv(index=False))
```

Validaciones semánticas: el número de columnas debe coincidir con el número de valores. Cada columna debe existir en el archivo CSV.

ACTUALIZAR — Modificar registros existentes

```
-- Sintaxis:
ACTUALIZAR "archivo.csv" VALORES col1=val1, col2=val2 DONDE condicion;

-- Ejemplo:
ACTUALIZAR "datos.csv" VALORES salario=65000, departamento="Arquitectura"
DONDE nombre = "Carlos Mendoza";

-- Python generado:
df = pd.read_csv("datos.csv")
condicion = (df["nombre"] == "Carlos Mendoza")
df.loc[condicion, "salario"] = 65000
df.loc[condicion, "departamento"] = "Arquitectura"
df.to_csv("datos.csv", index=False)
print(df.to_csv(index=False))
```

Validaciones semánticas: número de columnas == número de valores. Todas las columnas deben existir. La condición DONDE es obligatoria y debe pasar la validación semántica.

ELIMINAR — Borrar registros

```
-- Sintaxis:
ELIMINAR "archivo.csv" DONDE condicion;

-- Ejemplo:
ELIMINAR "datos.csv" DONDE edad < 25;

-- Python generado:
df = pd.read_csv("datos.csv")
condicion = (df["edad"] < 25)
df = df.loc[~condicion]
df.to_csv("datos.csv", index=False)
print(df.to_csv(index=False))
```

La cláusula DONDE es obligatoria en ELIMINAR para evitar borrado accidental de todos los registros. El operador ~ invierte la condición en Pandas (mantiene las filas que NO cumplen la condición).

Catálogo de Consultas de Ejemplo

A continuación, se presentan todas las consultas que el lenguaje soporta, organizadas por categoría. Cada una incluye una descripción de su propósito.

Consultas de Lectura (TRAER)

- **Todas las columnas:**

```
TRAER * DESDE "datos.csv";
```

- **Columnas específicas:**

```
TRAER nombre, edad, salario DESDE "datos.csv";
```

- **Con filtro igual:**

```
TRAER nombre, edad DESDE "datos.csv" DONDE nombre = "Carlos Mendoza";
```

- **Con filtro numérico:**

```
TRAER * DESDE "datos.csv" DONDE salario > 70000;
```

- **Con rango numérico (Y):**

```
TRAER nombre, edad DESDE "datos.csv" DONDE edad >= 30 Y edad <= 40;
```

- **Con operador diferente:**

```
TRAER * DESDE "datos.csv" DONDE departamento != "Ingeniería";
```

- **Agrupación lógica O:**

```
TRAER nombre DESDE "datos.csv" DONDE (ciudad = "Madrid" O ciudad = "Roma");
```

- **Agrupación lógica Y+O:**

```
TRAER * DESDE "datos.csv" DONDE salario > 50000 Y (departamento = "Ingeniería" O departamento = "Diseño");
```

- **Expresión aritmética:**

```
TRAER * DESDE "datos.csv" DONDE (salario * 0.2) < 10000;
```

- **Aritmética sin paréntesis:**

```
TRAER nombre, salario DESDE "datos.csv" DONDE salario - 5000 > 60000;
```

- **Subconsulta escalar:**

```
TRAER nombre, edad DESDE "datos.csv" DONDE edad > (TRAER edad DESDE "datos.csv" DONDE nombre = "Carlos Mendoza");
```

- **ORDENAR ASC:**

```
TRAER nombre, salario DESDE "datos.csv" ORDENAR POR salario ASC;
```

- **ORDENAR DESC:**

```
TRAER nombre, edad DESDE "datos.csv" ORDENAR POR edad DESC;
```

- **LIMITAR resultados:**

```
TRAER nombre, salario DESDE "datos.csv" ORDENAR POR salario DESC LIMITAR 5;
```

- **DISTINTO:**

```
TRAER DISTINTO departamento DESDE "datos.csv";
```

- **CONTAR:**
TRAER CONTAR(*) DESDE "datos.csv";
- **CONTAR con filtro:**
TRAER CONTAR(*) DESDE "datos.csv" DONDE departamento = "Ingeniería";
- **PROMEDIO:**
TRAER PROMEDIO(salario) DESDE "datos.csv";
- **SUMA:**
TRAER SUMA(salario) DESDE "datos.csv";
- **MAXIMO y MINIMO:**
TRAER MAXIMO(edad) DESDE "datos.csv";
- **GUARDAR EN:**
TRAER * DESDE "datos.csv" DONDE departamento = "Ingeniería" GUARDAR EN "resultado.csv";
- **Consulta compleja:**
TRAER nombre, edad, salario DESDE "datos.csv" DONDE salario >= 40000 Y salario <= 80000 ORDENAR POR edad ASC LIMITAR 8;

Operaciones CRUD

- **INSERTAR registro:**
INSERTAR EN "datos.csv" (id, nombre, edad, salario) VALORES (031, "Ana García", 27, 52000);
- **ACTUALIZAR un campo:**
ACTUALIZAR "datos.csv" VALORES salario=65000 DONDE nombre = "Carlos Mendoza";
- **ACTUALIZAR múltiples campos:**
ACTUALIZAR "datos.csv" VALORES salario=70000, departamento="Arquitectura" DONDE nombre = "Elena Rossi";
- **ELIMINAR por condición:**
ELIMINAR "datos.csv" DONDE edad < 25;
- **ELIMINAR por igualdad:**
ELIMINAR "datos.csv" DONDE departamento = "Temporal";

Interfaz de Usuario

La interfaz fue construida con JavaFX usando un layout dividido en dos paneles principales separados por un SplitPane vertical: el editor de código en la parte superior y los paneles de resultados en la inferior. El divisor es arrastrable para ajustar la proporción según la necesidad del usuario.

Panel Superior — Editor

El editor usa RichTextFX (CodeArea) con resaltado de sintaxis en tiempo real. Las palabras reservadas se colorean al escribir. La barra de estado inferior muestra el archivo CSV activo, el número de columnas detectadas, la codificación y la posición del cursor (línea, columna).

Panel Lateral — Árbol de Archivos

El panel lateral muestra el árbol de archivos CSV disponibles en el directorio de recursos. Al hacer clic en un archivo, su ruta se inserta automáticamente en el editor y la tabla de símbolos se precarga para acelerar la validación semántica.

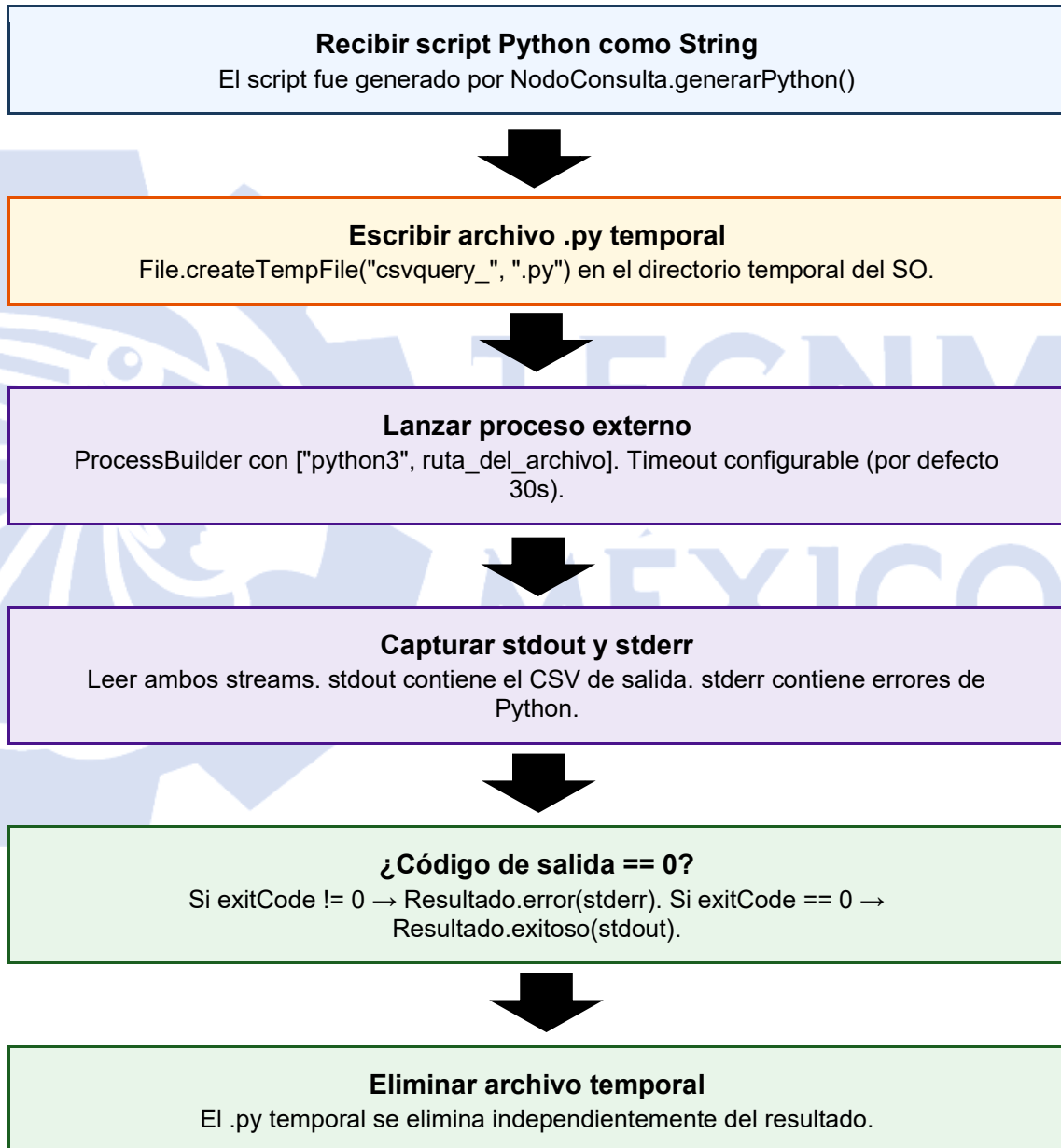
Pestañas de Resultado

Pestaña	Contenido
Resultado	TableView con el CSV devuelto por Pandas. Las columnas se generan dinámicamente.
Análisis Sintáctico	Árbol AST visual + tokens agrupados por cláusula (TRAER, DESDE, DONDE, etc.) con sus IDs.
Tokens	Tabla con ID numérico, nombre del token y lexema reconocido para cada token de la consulta.
Consola de Compilación	Terminal visual con fases del compilador. Entradas \$ (sistema), ✓ (OK), × ERR con badge de código.
Código Intermedio	Script Python/Pandas generado en un área de texto con tipografía monoespaciada (fondo oscuro).

Ejecución del Script Python

El script Python generado se ejecuta a través de la clase PandasRunner. Este componente escribe el script en un archivo .py temporal en el directorio del sistema, lanza un proceso externo con python3, captura la salida estándar y de error, y retorna un objeto Resultado con la salida o el mensaje de error.

Flujo de PandasRunner





Retornar Resultado al MainController

El controller muestra el CSV en la tabla o el error en la consola con código 5210.

Estructura del Proyecto

src/main/	
├─ java/org/example/csvquery/	
│ ├─ Launcher.java	→ Punto de entrada de la aplicación
│ ├─ MainApplication.java	→ Carga el FXML principal
│ └─ MainController.java	→ Controlador JavaFX – orquesta todo el
├─ pipeline	
│ ├─ Parser.java	→ Análisis sintáctico – construye el AST
│ ├─ PandasRunner.java	→ Ejecuta el script Python en proceso externo
│ └─ models/	
│ └─ Lexer.java	→ Análisis léxico con autómata cargado desde
├─ CSV	
│ ├─ Token.java	→ Modelo de dato: nombre, lexema, ID numérico
│ ├─ TablaSimbolosCSV.java	→ Carga y consulta tipos de columnas del CSV
│ └─ ast/	
│ ├─ NodoAST.java	→ Interfaz base del AST
│ └─ TipoDato.java	→ Enum: ENTERO, DECIMAL, CADENA, BOOLEANO,
├─ ERROR	
│ ├─ NodoInfo.java	→ Modelo para visualizar el árbol en la UI
│ ├─ NodoConsulta.java	→ Nodo raíz de consultas TRAER
│ ├─ NodoSeleccion.java	→ Cláusula TRAER (columnas / * / agregación)
│ ├─ NodoDesde.java	→ Cláusula DESDE (lectura del CSV)
│ ├─ NodoDonde.java	→ Cláusula DONDE (filtrado)
│ ├─ NodoOrdenar.java	→ Cláusula ORDENAR POR
│ ├─ NodoLimitar.java	→ Cláusula LIMITAR
│ ├─ NodoGuardar.java	→ Cláusula GUARDAR EN
│ ├─ NodoOperacionBinaria.java	→ Comparaciones y lógica Y/O
│ ├─ NodoExpresionAritmetica.java	→ Expresiones * + - /
│ ├─ NodoSubconsulta.java	→ Subconsultas escalares entre paréntesis
│ ├─ NodoAgregacion.java	→ Funciones CONTAR, SUMA, PROMEDIO, etc.
│ ├─ NodoColumna.java	→ Referencia a una columna del CSV
│ ├─ NodoLiteral.java	→ Valores literales (números, cadenas)
│ ├─ NodoInsertar.java	→ Operación INSERTAR EN
│ ├─ NodoActualizar.java	→ Operación ACTUALIZAR ... VALORES ... DONDE
│ └─ NodoEliminar.java	→ Operación ELIMINAR ... DONDE
└─ resources/	
│ ├─ org/example/csvquery/	
│ │ ├─ main-view.fxml	→ Layout principal de la interfaz
│ │ └─ css/csv-query.css	→ Estilos de la interfaz
│ └─ CSV/	
│ ├─ Matriz de transicion 2.csv	→ Autómata: 27 estados × alfabeto
│ └─ datos.csv	→ Archivo de datos de ejemplo

Nuestro analizador léxico lo implementamos mediante un autómata finito determinista, el cual va a ser el encargado de reconocer los diferentes tokens válidos del lenguaje. El autómata recibe como entrada una secuencia de caracteres proveniente de la consulta ingresada por el usuario y realiza transiciones entre estados dependiendo del símbolo leído.

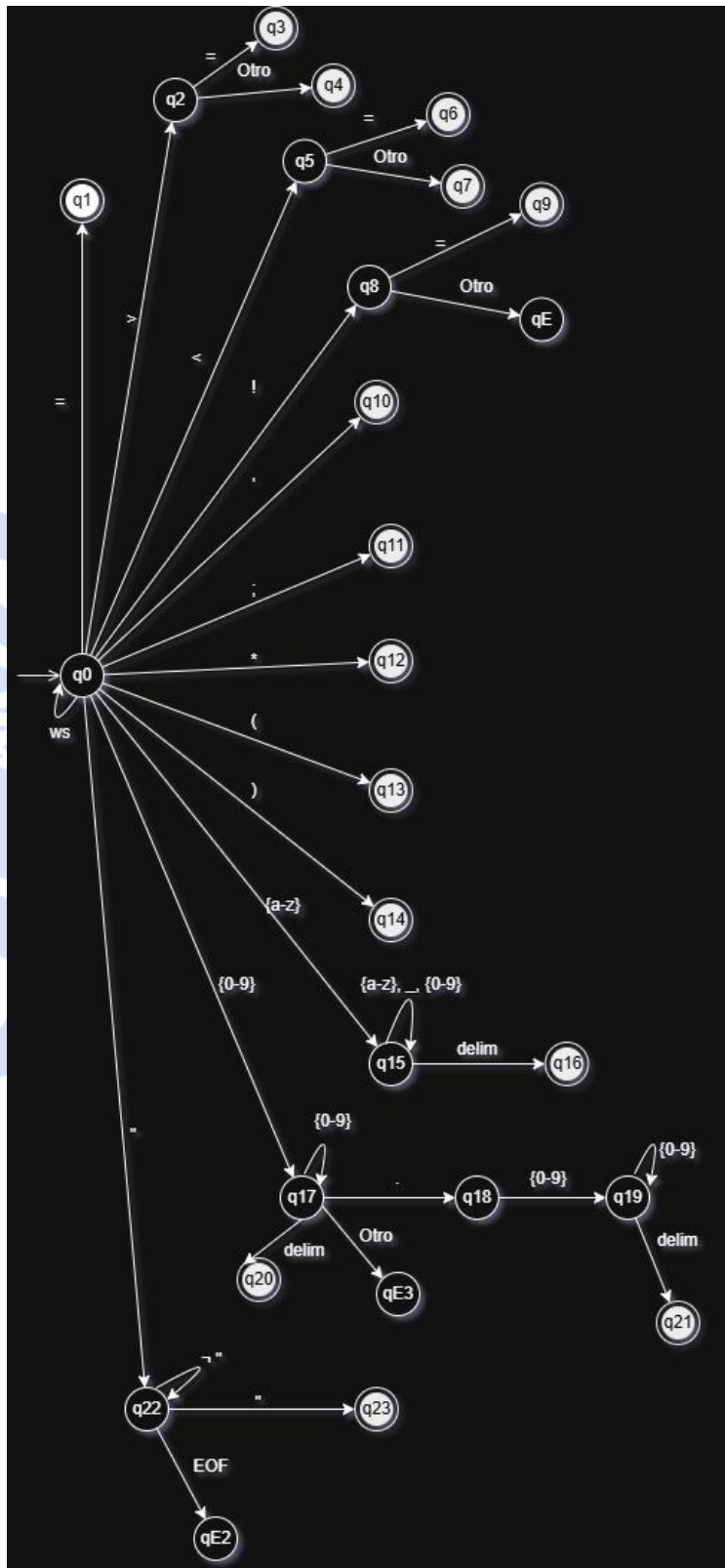
En el estado inicial de nuestro autómata corresponde a q0, que es donde vamos a evaluar diferentes tipos de caracteres, tales como:

- Operadores.
- Letras.
- Números.
- Delimitadores.
- Símbolos especiales.
- Cadenas de texto.

Cada una de las transiciones nos va a dirigir a un nuevo estado, el cual es capaz de reconocer un patrón ya antes definido, los estados de aceptación nos van a representar los tokens que vamos a considerar como válidos, algunos ejemplos que se pueden tomar en cuenta son:

- Operadores relacionales.
- Operadores aritméticos.
- Identificadores.
- Números enteros.
- Números flotantes.
- Cadenas.
- Paréntesis.
- Comas.
- Palabras reservadas.

Por otro lado, si nos damos cuenta y visualizamos con más detalle, nuestro autómata también tiene algunos errores léxicos (qE1), utilizados para detectar secuencias inválidas o formatos incorrectos durante el análisis.



La tabla de transiciones nos va a representar mas que nada el comportamiento de cómo va a funcionar el autómata anterior, ya que es el que vamos a utilizar en el análisis léxico. Cada una de las filas va a corresponder a un estado del autómata, mientras que cada columna representa un símbolo o categoría de los caracteres que colocamos en el alfabeto que va a ser reconocido por nuestro lenguaje.

Las celdas de la tabla nos van a indicar el siguiente estado al que debe de ir cuando el autómata se valla ejecutando, dependiendo del estado actual y del carácter que se haya leído. De esta manera, el Lexer puede recorrer dinámicamente la matriz para reconocer tokens válidos carácter por carácter.

La tabla incluye transiciones para:

- Operadores relacionales ($=$, $>$, $<$, $!=$, $>=$, $<=$)
- Operadores aritméticos ($+$, $-$, $*$, $/$)
- Identificadores
- Números enteros y flotantes
- Cadenas de texto
- Paréntesis
- Delimitadores
- Espacios en blanco
-

Además, se contemplan estados especiales para:

- Aceptación de tokens,
- Delimitación de lexemas,
- Detección de errores léxicos,
- Manejo de cadenas,
- Reconocimiento de fin de archivo (EOF).
-

La matriz de transición fue almacenada en un archivo CSV y posteriormente cargada dinámicamente por el Lexer durante la ejecución del compilador, permitiendo una implementación flexible y modular del análisis léxico.

[illegible]

REVISIONES

PRIMERA REVISION: 13 DE ABRIL DEL 2026

Durante la primera revisión del proyecto el equipo trabajo principalmente en la planeación del sistema, y en el diseño de la interfaz grafica del compilador, utilizando JavaFX. El objetivo principal fue crear un entorno amigable para los usuarios que los permitan escribir las consultas, visualizar resultados y mostrar errores de compilación. En esta etapa se definieron componentes importantes de la interfaz como el editor de consultas, las tablas de resultados, las ventanas de tokens y las áreas destinadas a la consola de compilación.

Además, se realizo una investigación sobre la integración entre Java, Python y la librería Pandas. Se analizaron distintas alternativas para poder ejecutar script de Python desde Java y así poder manipular archivos CSV. Esta investigación nos ayudó a decidir cómo armar el proyecto. Al final, elegimos que el compilador cree código Python automáticamente para hacer las consultas con Pandas. Así nos evitamos complicaciones al manejar las tablas y aprovechamos todo lo que ofrece esa librería

SEGUNDA REVISION: 28 DE ABRIL DEL 2026

En la segunda revisión empezamos bien con el desarrollo del analizador léxico. Lo primero que hicimos fue construir el autómata (AFD) para que pudiera reconocer los tokens. Diseñamos los diferentes estados y los caminos necesarios para identificar operadores, números, nombres de variables, palabras reservadas, textos y símbolos especiales de CSV-Query.

Después, pasamos esa lógica a una tabla de transiciones usando una matriz guardada en un archivo CSV. Esto nos sirvió para representar el comportamiento del autómata y ayudó a que el programa cargara los estados de forma dinámica. Con este cambio, el sistema ya pudo revisar las consultas carácter por carácter y empezó a separar bien las palabras. En esta misma etapa terminamos la primera versión funcional del analizador y logramos que mostrara la tabla de tokens en la pantalla del programa. Así pudimos comprobar que el autómata de verdad reconocía las palabras reservadas, los operadores y los demás componentes básicos del lenguaje.

TERCERA REVISION: 30 DE ABRIL DEL 2026

En la tercera revisión dejamos listo el analizador léxico agregando más validaciones y el manejo de errores. Con esto, el sistema empezó a detectar problemas como caracteres que no pertenecen al lenguaje, operadores incompletos, textos sin cerrar y números mal escritos. También creamos un sistema con códigos de error para que fuera más fácil encontrar las fallas durante la compilación.

En cuanto terminamos con el lexer, empezamos a trabajar en el análisis sintáctico usando un parser descendente recursivo. Diseñamos este parser para revisar que el orden de los tokens cumpliera al pie de la letra con las reglas de la gramática que definimos para CSV-Query.

Además, empezamos a armar el Árbol Sintáctico Abstracto (AST), que es la estructura en forma de árbol que usamos para representar internamente cada consulta. En esta etapa creamos los nodos para las consultas SELECT, las condiciones lógicas, las operaciones y los diferentes tipos de datos. Usar el AST nos sirvió muchísimo para separar bien las etapas del compilador y hacer que el análisis de las consultas fuera más sencillo

CUARTA REVISION: 13 DE MAYO DEL 2026

En la cuarta revisión trabajamos en el análisis semántico del compilador. Esta etapa nos sirvió para revisar que las consultas no solo tuvieran un buen orden, sino que también tuvieran sentido lógico y estructural. Agregamos validaciones para checar que las columnas de verdad existieran en los archivos CSV, que los tipos de datos fueran compatibles entre sí y que las operaciones matemáticas y lógicas estuvieran bien hechas.

Durante esta fase también hicimos un montón de pruebas usando operadores lógicos como AND que lo representamos como "Y" y OR que lo representamos como "O", además de los signos de operaciones y comparaciones. Estas pruebas nos ayudaron a comprobar que las condiciones complejas dentro del WHERE funcionaran bien y que el código Python se estuviera creando correctamente desde el AST.

Además, mejoramos la conexión entre Java y Python, lo que nos permitió correr automáticamente los scripts que se iban creando y mostrar los resultados directo en la pantalla del programa.

QUINTA REVISION: 21 DE MAYO DEL 2026

En la quinta revisión agregamos nuevas funciones para poder usar operaciones matemáticas y agrupaciones con paréntesis. Con esto logramos que CSV-Query soporte consultas mucho más completas, dando más libertad al momento de armar la lógica y las operaciones.

Hicimos mejoras importantes en el parser para que pudiera reconocer bien las operaciones agrupadas y las fórmulas que van una dentro de otra. También optimizamos la forma en que se arma el AST y cómo se traduce todo a código de Python con Pandas.

Además, creamos la sección 'About' (Acerca de) en la pantalla del programa, donde pusimos información sobre quiénes desarrollamos el proyecto, una explicación general del sistema y el acceso a la documentación. Esto le dio una vista más profesional al proyecto y ayuda a que cualquiera que lo use entienda rápido de qué trata la aplicación.

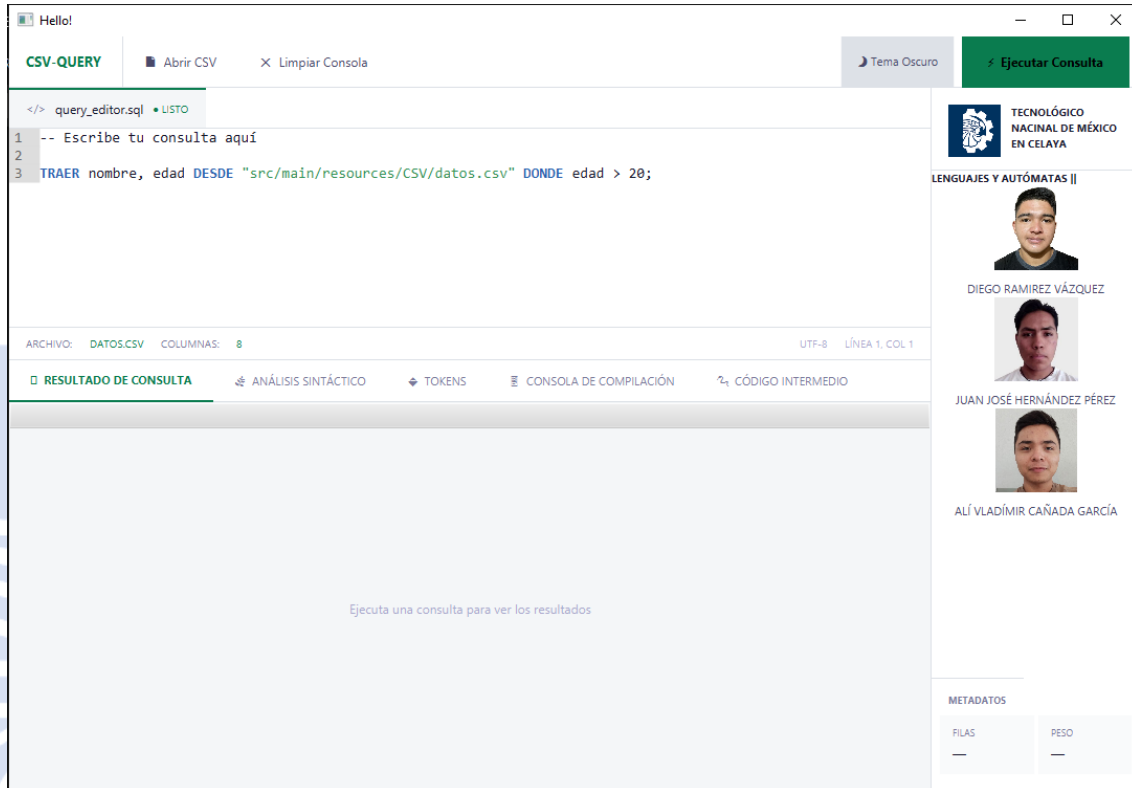
SEXTA REVISION: 25 DE MAYO DEL 2026

En la sexta y última revisión agregamos las funciones que faltaban para terminar el modelo CRUD en CSV-Query. En esta etapa creamos las operaciones INSERT, UPDATE y DELETE, lo que ya permite meter nuevos datos, actualizar los que ya existen y borrar información directamente usando las consultas del lenguaje.

Para lograrlo, tuvimos que hacer más grande la gramática del parser, crear nuevos nodos en el AST y agregar validaciones semánticas para asegurar que los cambios de datos se hicieran bien. También programamos la generación automática de código Python para que Pandas modifique los DataFrames y al final guarde los cambios de vuelta en los archivos CSV.

Para cerrar, hicimos pruebas completas a todo el sistema para revisar que cada etapa del compilador (el léxico, el sintáctico y el semántico) funcionara al cien, además de comprobar que las consultas SELECT, INSERT, UPDATE y DELETE corrieran sin problemas. Estas pruebas nos sirvieron para confirmar que el programa es estable y que cumplimos con todo lo que nos propusimos al inicio del proyecto

INICIO DE INTERFAZ

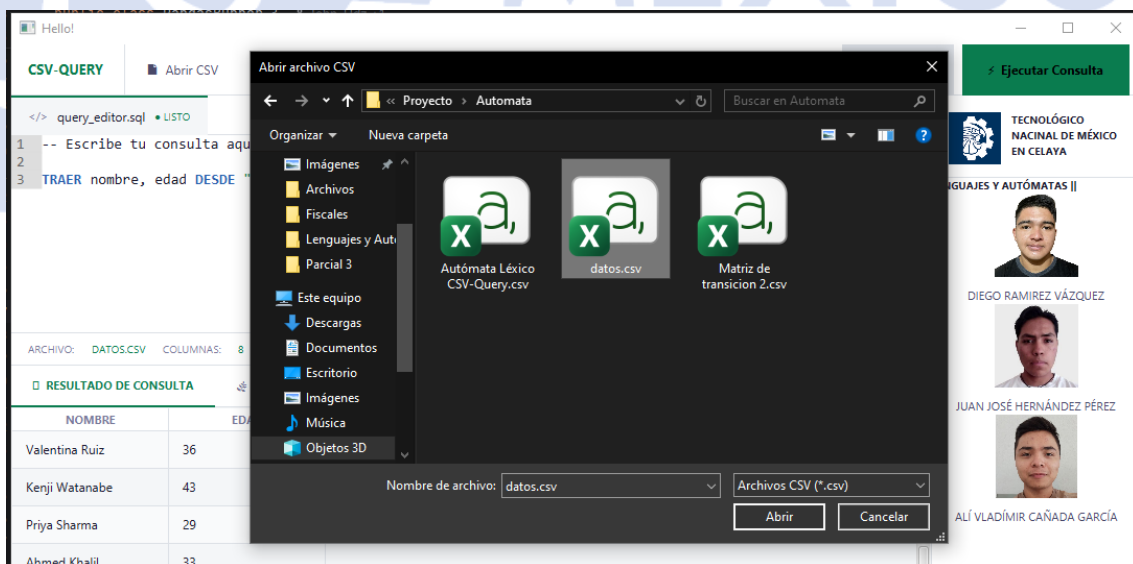


The screenshot shows the CSV-QUERY application interface. The main window has a title bar with "Hello!". Below the title bar, there's a menu bar with "CSV-QUERY", "Abrir CSV", and "Limpiar Consola". On the right, there's a "Tema Oscuro" button and a green "Ejecutar Consulta" button. The main area is a query editor with a text area containing the following SQL query:

```
</> query_editor.sql • LISTO
1 -- Escribe tu consulta aquí
2
3 TRAER nombre, edad DESDE "src/main/resources/CSV/datos.csv" DONDE edad > 20;
```

Below the query editor, there's a status bar showing "ARCHIVO: DATOS.CSV COLUMNAS: 8" and "UTF-8 LÍNEA 1, COL 1". Below the status bar, there's a tabbed interface with "RESULTADO DE CONSULTA" selected. The result area is empty, showing "Ejecuta una consulta para ver los resultados". On the right, there's a sidebar with the TECNOLÓGICO NACIONAL DE MÉXICO EN CELAYA logo, a section "LENGUAJES Y AUTÓMATAS II" with three user avatars (DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALÍ VLADÍMIR CAÑADA GARCÍA), and a "METADATOS" section with "FILAS" and "PESO" labels.

ABIR CSV



The screenshot shows the CSV-QUERY application interface with the "Abrir archivo CSV" dialog box open. The dialog box is titled "Abrir archivo CSV" and shows the file selection process. The file list includes "Autómata Léxico CSV-Query.csv", "datos.csv", and "Matriz de transición 2.csv". The "Nombre de archivo" field is set to "datos.csv" and the file type is "Archivos CSV (*.csv)". The "Abrir" button is highlighted. In the background, the CSV-QUERY application is visible, showing the same query editor and sidebar as in the previous screenshot. The result area now displays a table with the following data:

NOMBRE	EDAD
Valentina Ruiz	36
Kenji Watanabe	43
Priya Sharma	29
Ahmed Khalil	33

RESULTADOS DE EJECUCIÓN

ARCHIVO: DATOS.CSV COLUMNAS: 8		UTF-8 LÍNEA 1, COL 1	
RESULTADO DE CONSULTA		ANÁLISIS SINTÁCTICO	TOKENS
CONSOLA DE COMPILACIÓN		CÓDIGO INTERMEDIO	
NOMBRE	EDAD		
Valentina Ruiz	36		
Kenji Watanabe	43		
Priya Sharma	29		
Ahmed Khalil	33		
Sofia Petrov	41		
Diego Morales	24		
Nadia Boucher	38		
Kwame Asante	30		
Isabella Costa	47		
Ravi Patel	26		



JUAN JOSÉ HERNÁNDEZ PÉREZ



ALÍ VLADÍMIR CAÑADA GARCÍA

METADATOS

FILAS	PESO
30	—

ANÁLISIS SINTACTICO

ARCHIVO: DATOS.CSV

COLUMNAS: 8

UTF-8 LÍNEA 1, COL 1

RESULTADO DE CONSULTA

ANÁLISIS SINTÁCTICO

TOKENS

CONSOLA DE COMPILACIÓN

CÓDIGO INTERMEDIO

11 tokens

3 cláusulas

SÍ AST válido

0 errores

Árbol de Sintaxis Abstracta (AST)

NodoConsulta

consulta completa

NodoSelección

nombre, edad

NodoColumna

nombre

NodoColumna

edad

NodoDesde

"src/main/resources/CSV/datos.csv"

NodoDonde

condición de filtrado

NodoOperacionBinaria

>

NodoColumna

edad

NodoLiteral

20 (ENTERO)

NodoOrdenar

— no definido

NodoLimitar

— no definido

NodoGuardar

— no definido

Tokens agrupados por cláusula

TRAER ✓ OK

Columnas o función de agregación a obtener

TRAER

nombre

,

edad

TRAER

ID

COMA

ID

DESDE ✓ OK

Archivo CSV de origen de los datos

DESDE

"src/main/resources/CSV/datos.csv"

DESDE

STRING

DONDE ✓ OK

Condición de filtrado sobre los registros

DONDE

edad

>

20

;

DONDE

ID

MAYOR

ENTERO

PUNTOCOMA

JUAN JOSÉ HERNÁNDEZ PÉREZ

ALÍ VLADÍMIR CAÑADA GARCÍA

METADATOS

FILAS

PESO

30

—

TABLA DE TOKENS

ARCHIVO: DATOS.CSV COLUMNAS: 8		UTF-8 LÍNEA 1, COL 1		JUAN JOSÉ HERNÁNDEZ PÉREZ	
RESULTADO DE CONSULTA		ANÁLISIS SINTÁCTICO		TOKENS	
CONSOLA DE COMPILACIÓN		CÓDIGO INTERMEDIO		ALÍ VLADÍMIR CAÑADA GARCÍA	
ID	TOKEN	LEXEMA			
2001	TOKEN_TRAER	TRAER			
4001	TOKEN_ID	nombre			
3001	TOKEN_COMA	,			
4001	TOKEN_ID	edad			
2002	TOKEN_DESDE	DESDE			
4004	TOKEN_STRING	"src/main/resources/CSV/d...			
2003	TOKEN_DONDE	DONDE			
4001	TOKEN_ID	edad			
1002	TOKEN_MAYOR	>			
4002	TOKEN_ENTERO	20			
3002	TOKEN_PUNTOCOMA	;			

METADATOS

FILAS
30

PESO
—

CONSOLA DE COMPILACIÓN

```

CONSOLA DE COMPILACIÓN

$ Iniciando análisis léxico...
✓ Autómata cargado — 11 token(s) reconocido(s)

$ Iniciando análisis sintáctico...
✓ AST construido correctamente

$ Iniciando análisis semántico...
✓ Validación semántica exitosa

$ Generando código intermedio...
✓ Script Python generado — 7 línea(s)

$ Ejecutando script con Pandas...
✓ Consulta ejecutada con éxito
  
```

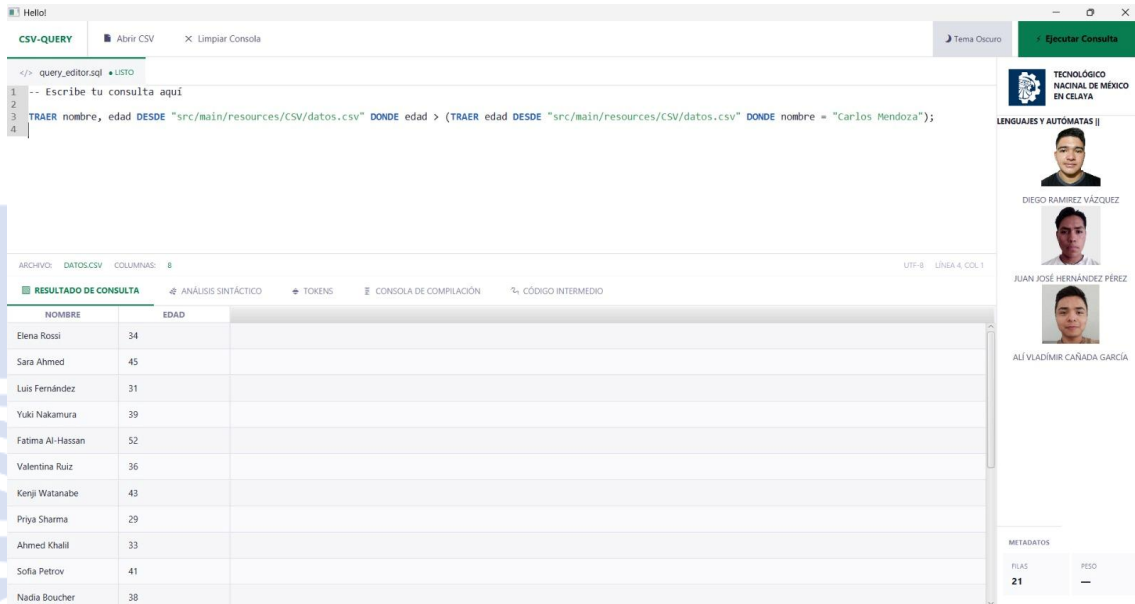
CODIGO INTERMEDIO

ARCHIVO: DATOS.CSV COLUMNAS: 8		UTF-8 LÍNEA 1, COL 1		JUAN JOSÉ HERNÁNDEZ PÉREZ	
RESULTADO DE CONSULTA		ANÁLISIS SINTÁCTICO		CÓDIGO INTERMEDIO	
CONSOLA DE COMPILACIÓN		CÓDIGO INTERMEDIO GENERADO		ALÍ VLADÍMIR CAÑADA GARCÍA	
		Python / Pandas			
		<pre> import pandas as pd import sys df = pd.read_csv(r"src/main/resources/CSV/datos.csv") df = df[(df["edad"] > 20)] df = df[["nombre", "edad"]] print(df.to_csv(index=False)) </pre>			

PRUEBAS FINALES.

Se realizaron varias pruebas en las cuales nos tiene que ejecutar de manera correcta.

- TRAER nombre, edad DESDE "src/main/resources/CSV/datos.csv" DONDE edad > (TRAER edad DESDE "src/main/resources/CSV/datos.csv" DONDE nombre = "Carlos Mendoza");



The screenshot shows the Helio IDE interface. The query editor contains the following SQL query:

```

1  -- Escribe tu consulta aquí
2
3  TRAER nombre, edad DESDE "src/main/resources/CSV/datos.csv" DONDE edad > (TRAER edad DESDE "src/main/resources/CSV/datos.csv" DONDE nombre = "Carlos Mendoza");
4

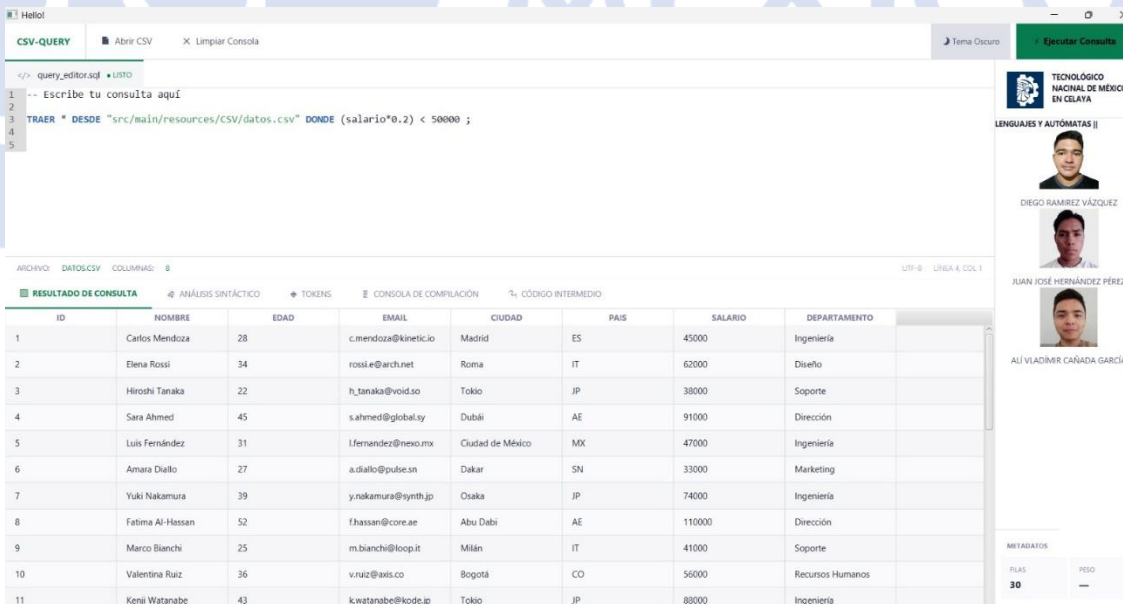
```

The results pane displays a table with the following data:

NOMBRE	EDAD
Elena Rossi	34
Sara Ahmed	45
Luis Fernández	31
Yuki Nakamura	39
Fatima Al-Hassan	52
Valentina Ruiz	36
Kenji Watanabe	43
Priya Sharma	29
Ahmed Khalil	33
Sofia Petrov	41
Nadia Boucher	38

On the right side, there is a sidebar with the logo of the Tecnológico Nacional de México en Celaya and a list of users: DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALÍ VLADÍMIR CAÑADA GARCÍA.

- TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE (salario*0.2) < 50000 ;



The screenshot shows the Helio IDE interface. The query editor contains the following SQL query:

```

1  -- Escribe tu consulta aquí
2
3  TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE (salario*0.2) < 50000 ;
4
5

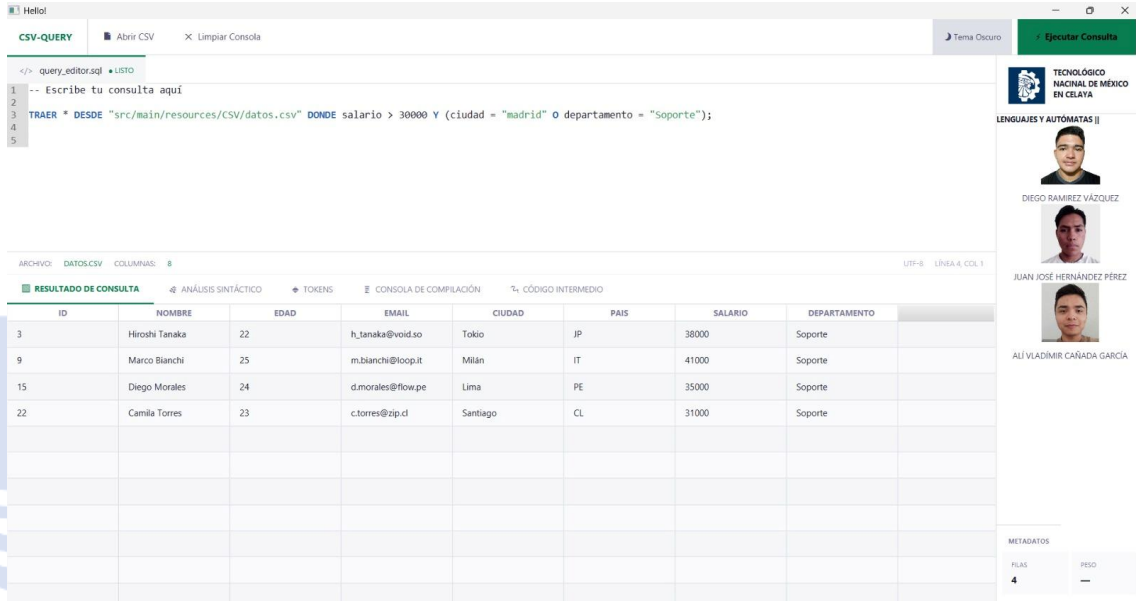
```

The results pane displays a table with the following data:

ID	NOMBRE	EDAD	EMAIL	CIUDAD	PAIS	SALARIO	DEPARTAMENTO
1	Carlos Mendoza	28	c.mendoza@kineticio	Madrid	ES	45000	Ingeniería
2	Elena Rossi	34	rossi.e@arch.net	Roma	IT	62000	Diseño
3	Hiroshi Tanaka	22	h.tanaka@void.so	Tokio	JP	38000	Soporte
4	Sara Ahmed	45	s.ahmed@global.sy	Dubái	AE	91000	Dirección
5	Luis Fernández	31	l.fernandez@neco.mx	Ciudad de México	MX	47000	Ingeniería
6	Amara Diallo	27	a.diallo@pulse.sn	Dakar	SN	33000	Marketing
7	Yuki Nakamura	39	y.nakamura@synth.jp	Osaka	JP	74000	Ingeniería
8	Fatima Al-Hassan	52	f.hassan@core.ae	Abu Dabí	AE	110000	Dirección
9	Marco Bianchi	25	m.bianchi@loop.it	Milán	IT	41000	Soporte
10	Valentina Ruiz	36	v.ruiz@axis.co	Bogotá	CO	56000	Recursos Humanos
11	Kenji Watanabe	43	k.watanabe@kode.jp	Tokio	JP	80000	Ingeniería

On the right side, there is a sidebar with the logo of the Tecnológico Nacional de México en Celaya and a list of users: DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALÍ VLADÍMIR CAÑADA GARCÍA.

- TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE salario > 30000 Y (ciudad = "madrid" O departamento = "Soporte");



CSV-QUERY

query_editor.sql • LISTO

```
-- Escribe tu consulta aquí
1
2
3 TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE salario > 30000 Y (ciudad = "madrid" O departamento = "Soporte");
4
5
```

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 4, COL 1

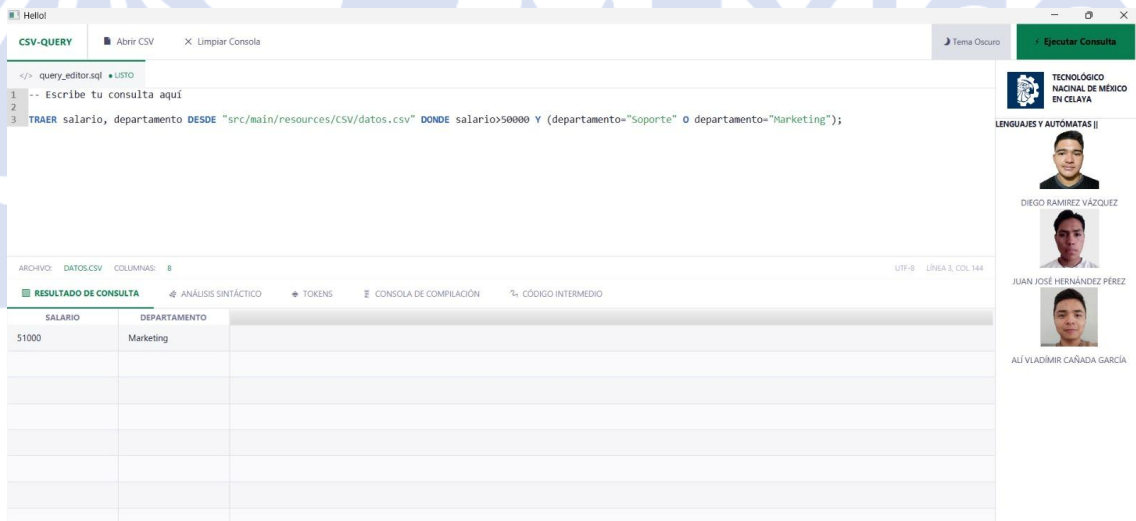
RESULTADO DE CONSULTA

ID	NOMBRE	EDAD	EMAIL	CIUDAD	PAIS	SALARIO	DEPARTAMENTO
3	Hiroshi Tanaka	22	h.tanaka@void.so	Tokio	JP	38000	Soporte
9	Marco Bianchi	25	m.bianchi@loop.it	Milán	IT	41000	Soporte
15	Diego Morales	24	d.morales@flow.pe	Lima	PE	35000	Soporte
22	Camila Torres	23	c.torres@zip.cl	Santiago	CL	31000	Soporte

METADATOS

FILAS: 4 PESO: —

- TRAER salario, departamento DESDE "src/main/resources/CSV/datos.csv" DONDE salario>50000 Y (departamento="Soporte" O departamento="Marketing");



CSV-QUERY

query_editor.sql • LISTO

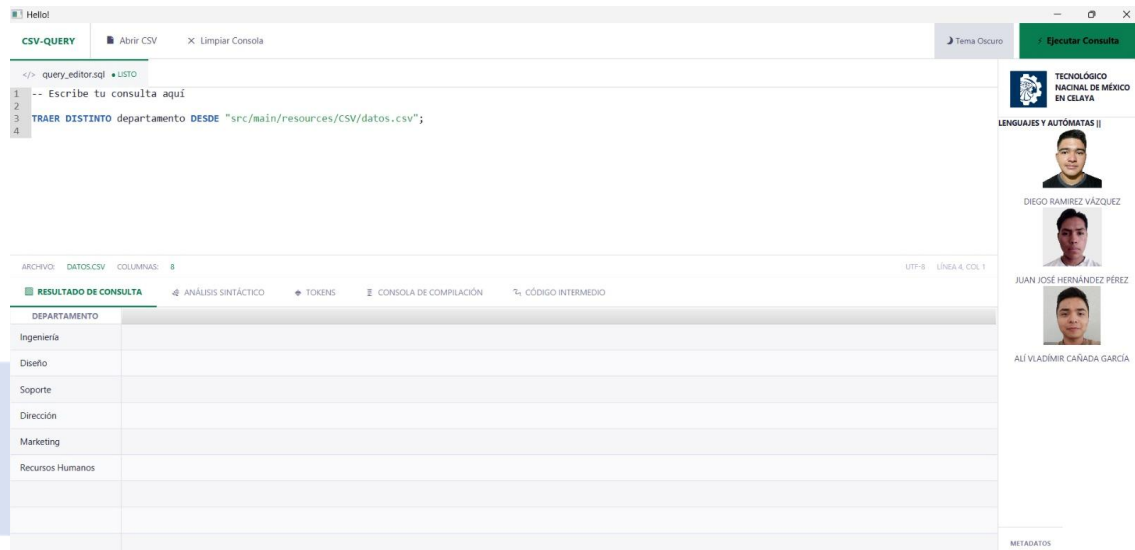
```
-- Escribe tu consulta aquí
1
2
3 TRAER salario, departamento DESDE "src/main/resources/CSV/datos.csv" DONDE salario>50000 Y (departamento="Soporte" O departamento="Marketing");
```

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 3, COL 144

RESULTADO DE CONSULTA

SALARIO	DEPARTAMENTO
51000	Marketing

- TRAER DISTINTO departamento DESDE "src/main/resources/CSV/datos.csv";



The screenshot shows the CSV-QUERY application interface. The query editor contains the following SQL query:

```

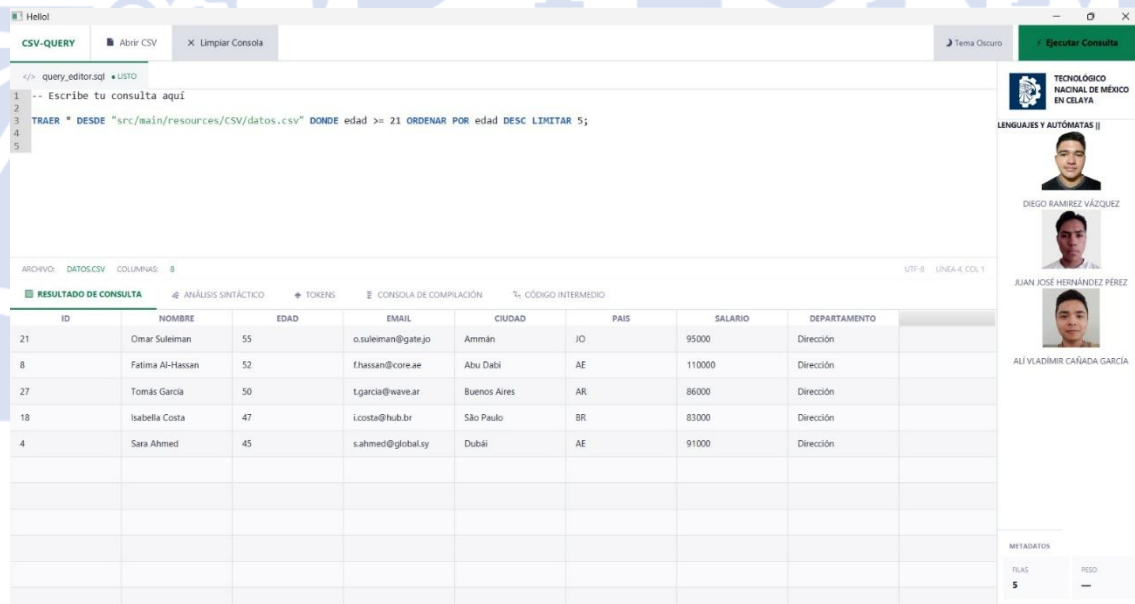
1  -- Escribe tu consulta aquí
2  TRAER DISTINTO departamento DESDE "src/main/resources/CSV/datos.csv";
3
4

```

The results pane shows a table with the following data:

DEPARTAMENTO
Ingeniería
Diseño
Soporte
Dirección
Marketing
Recursos Humanos

- TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE edad >= 21 ORDENAR POR edad DESC LIMITAR 5;



The screenshot shows the CSV-QUERY application interface. The query editor contains the following SQL query:

```

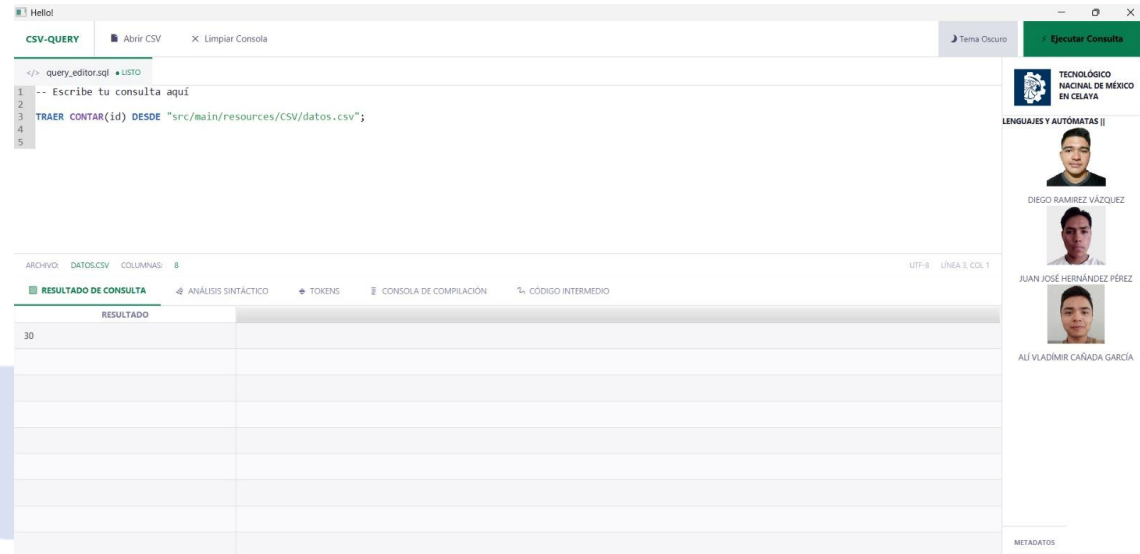
1  -- Escribe tu consulta aquí
2  TRAER * DESDE "src/main/resources/CSV/datos.csv" DONDE edad >= 21 ORDENAR POR edad DESC LIMITAR 5;
3
4
5

```

The results pane shows a table with the following data:

ID	NOMBRE	EDAD	EMAIL	CIUDAD	PAIS	SALARIO	DEPARTAMENTO
21	Omar Suleiman	55	o.suleiman@gatejo	Ammán	JO	95000	Dirección
8	Fatima Al-Hassan	52	f.hassan@core.ae	Abu Dabi	AE	110000	Dirección
27	Tomás García	50	tgarcia@wave.ar	Buenos Aires	AR	86000	Dirección
18	Isabella Costa	47	lcosta@hub.br	São Paulo	BR	83000	Dirección
4	Sara Ahmed	45	s.ahmed@global.sy	Dubai	AE	91000	Dirección

- TRAER CONTAR(id) DESDE "src/main/resources/CSV/datos.csv";



The screenshot shows the Helio IDE interface. The top bar includes 'CSV-QUERY', 'Abrir CSV', and 'Limpiar Consola'. The main editor contains the following SQL query:

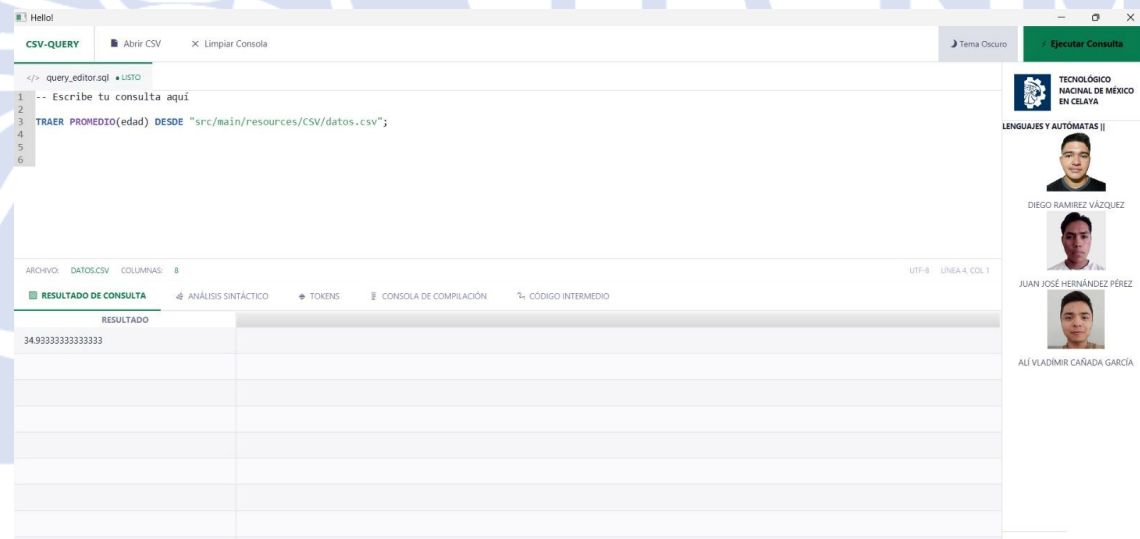
```

1  -- Escribe tu consulta aquí
2
3  TRAER CONTAR(id) DESDE "src/main/resources/CSV/datos.csv";
4
5

```

The right sidebar shows the 'Ejecutar Consulta' button and a list of users: DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALI VLADÍMIR CAÑADA GARCÍA. The bottom panel displays the 'RESULTADO DE CONSULTA' with a single row containing the value '30'.

- TRAER PROMEDIO(edad) DESDE "src/main/resources/CSV/datos.csv";



The screenshot shows the Helio IDE interface. The top bar includes 'CSV-QUERY', 'Abrir CSV', and 'Limpiar Consola'. The main editor contains the following SQL query:

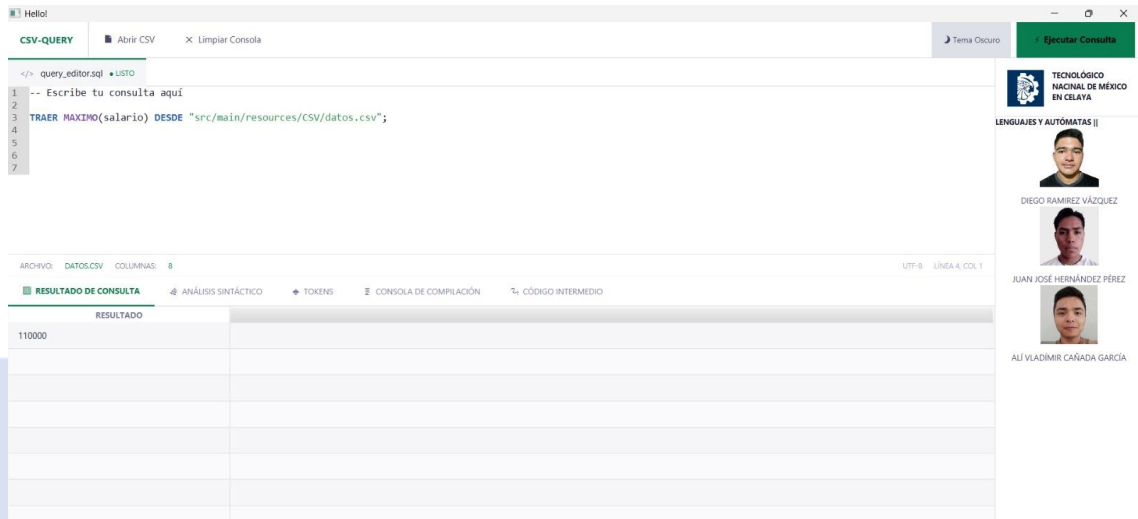
```

1  -- Escribe tu consulta aquí
2
3  TRAER PROMEDIO(edad) DESDE "src/main/resources/CSV/datos.csv";
4
5
6

```

The right sidebar shows the 'Ejecutar Consulta' button and the same list of users. The bottom panel displays the 'RESULTADO DE CONSULTA' with a single row containing the value '34.93333333333333'.

- TRAER MAXIMO(salario) DESDE "src/main/resources/CSV/datos.csv";



The screenshot shows the CSV-QUERY application interface. The query editor contains the following SQL query:

```

1 -- Escribe tu consulta aqui
2
3 TRAER MAXIMO(salario) DESDE "src/main/resources/CSV/datos.csv";
4
5
6
7

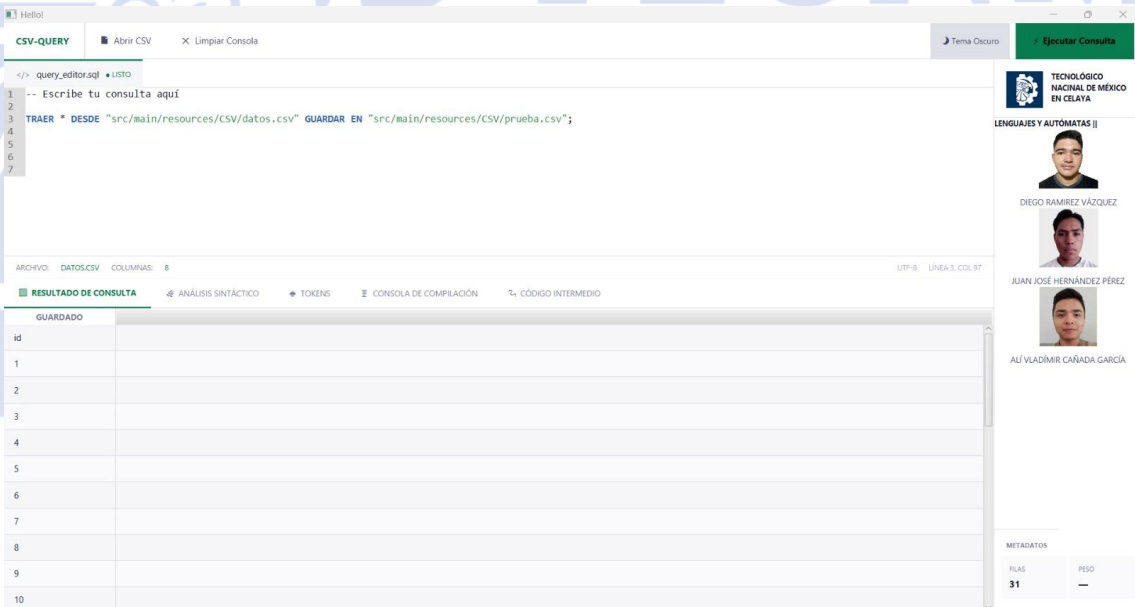
```

The results pane shows a single row with the value 110000.

RESULTADO
110000

On the right side, there is a sidebar with the TECNOLÓGICO NACIONAL DE MÉXICO EN CELAYA logo and a list of users: DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALÍ VLADIMIR CAÑADA GARCÍA.

- TRAER * DESDE "src/main/resources/CSV/datos.csv" GUARDAR EN "src/main/resources/CSV/prueba.csv";



The screenshot shows the CSV-QUERY application interface. The query editor contains the following SQL query:

```

1 -- Escribe tu consulta aqui
2
3 TRAER * DESDE "src/main/resources/CSV/datos.csv" GUARDAR EN "src/main/resources/CSV/prueba.csv";
4
5
6
7

```

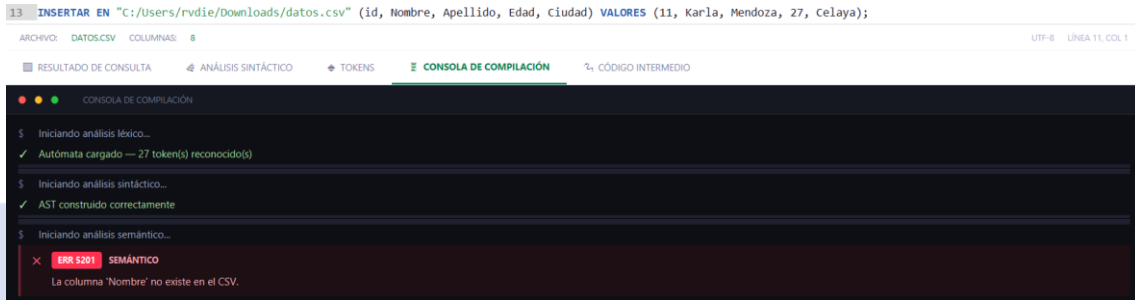
The results pane shows a table with 10 rows and 1 column (id).

GUARDADO
id
1
2
3
4
5
6
7
8
9
10

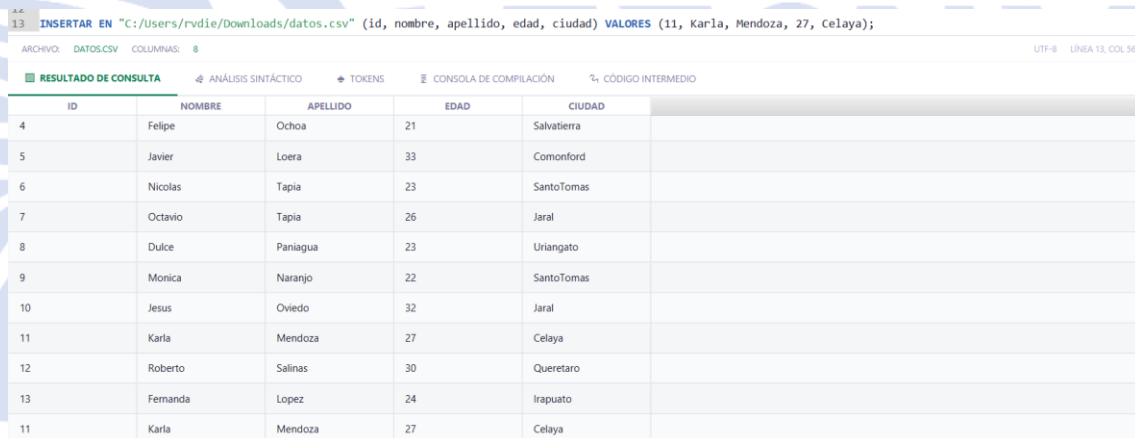
On the right side, there is a sidebar with the TECNOLÓGICO NACIONAL DE MÉXICO EN CELAYA logo and a list of users: DIEGO RAMÍREZ VÁZQUEZ, JUAN JOSÉ HERNÁNDEZ PÉREZ, and ALÍ VLADIMIR CAÑADA GARCÍA. At the bottom right, there is a METADATOS section showing 31 rows and 1 column.

La siguiente consulta nos va a fallar ya que siempre todas las columnas cuando queremos insertar tienen que tener el mismo nombre que en el CSV, entonces nos marcara error ya que en el CSV están todas en minúscula.

- INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, Nombre, Apellido, Edad, Ciudad) VALORES (11, Karla, Mendoza, 27, Celaya);



En cambio ya si cambiamos todo a minúscula, podemos ver que se agregan de manera correcta.



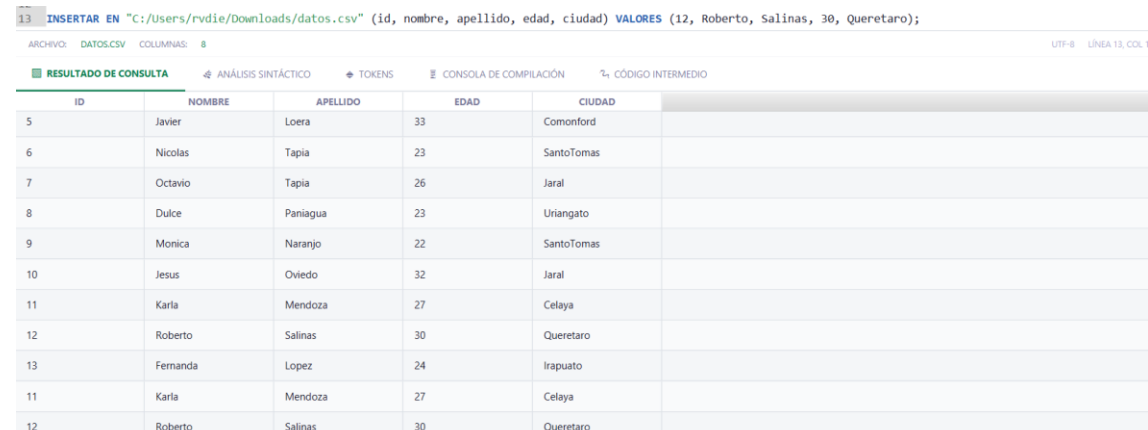
13 INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (11, Karla, Mendoza, 27, Celaya);

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 13, COL 56

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
4	Felipe	Ochoa	21	Salvatierra
5	Javier	Loera	33	Comonford
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro
13	Fernanda	Lopez	24	Irapuato
11	Karla	Mendoza	27	Celaya

- INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (12, Roberto, Salinas, 30, Queretaro);



13 INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (12, Roberto, Salinas, 30, Queretaro);

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 13, COL 1

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
5	Javier	Loera	33	Comonford
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro
13	Fernanda	Lopez	24	Irapuato
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro

En esta siguiente falla debido a que le estamos dando 5 atributos y solamente 3 valores entonces nos va a marcar error hasta que la verifiquemos

- INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (13, Fernanda, Lopez);



- INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (13, Fernanda, Lopez, 24, Irapuato);

13 INSERTAR EN "C:/Users/rvdie/Downloads/datos.csv" (id, nombre, apellido, edad, ciudad) VALORES (13, Fernanda, Lopez, 24, Irapuato);

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 13, COL 1

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro
13	Fernanda	Lopez	24	Irapuato
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro
13	Fernanda	Lopez	24	Irapuato

- ACTUALIZAR "C:/Users/rvdie/Downloads/datos.csv" VALORES edad = 35 DONDE (nombre = Javier);
Antes de la ejecución

5	Javier	Loera	33	Comonford
---	--------	-------	----	-----------

Ejecución.

3 ACTUALIZAR "src/main/resources/CSV/datos1.csv" VALORES edad = 35 DONDE (nombre = Javier);

ARCHIVO: DATOS.CSV COLUMNAS: 8 UTF-8 LÍNEA 3, COL 82

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
4	Felipe	Ochoa	21	Salvatierra
5	Javier	Loera	35	Comonford

- ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (edad = 24);
Antes de la ejecución:

13 ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (edad = 24);

ARCHIVO: DATOS.CSV COLUMNAS: 5 UTF-8 LÍNEA 12, COL 1

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro
13	Fernanda	Lopez	24	Irapuato

Ejecucion:

13 ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (edad = 24);

ARCHIVO: DATOS.CSV COLUMNAS: 5 UTF-8 LÍNEA 12, COL 1

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
4	Felipe	Ochoa	21	Salvatierra
5	Javier	Loera	33	Comonford
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya
12	Roberto	Salinas	30	Queretaro

- ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (ciudad = Celaya Y nombre = Karla);

Antes de la ejecución:

13 ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (ciudad = Celaya Y nombre = Karla);

ARCHIVO: DATOS.CSV COLUMNAS: 5 UTF-8 LÍNEA 13, COL 1

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
1	Diego	Ramirez	22	Jaral
2	Mayra	Fernandez	22	Queretaro
3	Manuel	Arrollo	42	Salvatierra
4	Felipe	Ochoa	21	Salvatierra
5	Javier	Loera	33	Comonford
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral
11	Karla	Mendoza	27	Celaya

Ejecucion:

13 ELIMINAR "C:/Users/rvdie/Downloads/datos.csv" DONDE (ciudad = Celaya Y nombre = Karla);

ARCHIVO: DATOS.CSV COLUMNAS: 5 UTF-8 LÍNEA 13, COL 89

RESULTADO DE CONSULTA ANÁLISIS SINTÁCTICO TOKENS CONSOLA DE COMPILACIÓN CÓDIGO INTERMEDIO

ID	NOMBRE	APELLIDO	EDAD	CIUDAD
1	Diego	Ramirez	22	Jaral
2	Mayra	Fernandez	22	Queretaro
3	Manuel	Arrollo	42	Salvatierra
4	Felipe	Ochoa	21	Salvatierra
5	Javier	Loera	33	Comonford
6	Nicolas	Tapia	23	SantoTomas
7	Octavio	Tapia	26	Jaral
8	Dulce	Paniagua	23	Uriangato
9	Monica	Naranjo	22	SantoTomas
10	Jesus	Oviedo	32	Jaral

9	VIDEO DE EXPLICACION DEL COMPILADOR
	LINK DEL VIDEO: https://youtu.be/UNeRV9BhoCg
10	CONCLUSIONES
	El desarrollo de CSV – Query nos permitió aplicar de forma practica todo lo que aprendimos sobre compiladores, lenguajes formales y manejo de datos. Al construir paso a paso el analizador léxico, sintáctico y semántico, pudimos entender de verdad cómo funciona un compilador por dentro y cómo es que logra traducir las instrucciones que escribe un usuario. Algo de los que mas nos agradó al final de este curso fue crear desde cero un lenguaje de consultas parecido a SQL que pudiera hacer operaciones CRUD (insertar, cambiar, borrar y consultar) directamente sobre archivos CSV. Con esto demostramos que se pueden usar archivos CSV como una opción ligera para manejar tablas de datos, sin tener que instalar un sistema de bases de datos completo y pesado Hacer un autómata y la tabla de transiciones nos ayudó y nos sirvió para ver cómo se podía implementar esto directamente al código de igual forma para dejar que todo el análisis léxico quedara funcionando como debe, reconociendo tokens. Después, armar el parser descendente recursivo y el Árbol Sintáctico Abstracto (AST) hizo que fuera mucho más fácil revisar que las consultas estuvieran bien estructuradas y representarlas dentro del programa. Además, conectar Java con Python y Pandas fue una gran solución para correr las consultas. En lugar de inventar la rueda, aprovechamos todo el poder que ya tiene Pandas para manejar tablas, haciendo que nuestro compilador genere código Python de forma automática. En general fue un proyecto muy interesante ya que así nos damos cuenta de como es el funcionamiento de un compilador que nosotros como programadores utilizamos día a día sin ni quiera saber como es que realiza las tareas según las instrucciones que nosotros como usuarios le estamos dando.

11	BIBLIOGRAFÍAS
	- Compiler design tutorial. (s/f). Tutorialspoint.com. Recuperado el 27 de mayo de 2026, de https://www.tutorialspoint.com/compiler_design/index.htm - Compiler design tutorial. (2024, junio 12). GeeksforGeeks. https://www.geeksforgeeks.org/compiler-design/compiler-design-tutorials/ - DataFrame — pandas 3.0.3 documentation. (s/f). Pydata.org. Recuperado el 27 de mayo de 2026, de https://pandas.pydata.org/docs/reference/frame.html - Java documentation. (2018, julio 30). Oracle Help Center. https://docs.oracle.com/en/java/ - Pandas documentation — pandas 3.0.3 documentation. (s/f). Pydata.org. Recuperado el 27 de mayo de 2026, de https://pandas.pydata.org/docs/ - Pandas tutorial. (s/f). W3schools.com. Recuperado el 27 de mayo de 2026, de https://www.w3schools.com/python/pandas/default.asp - Python 3.14 documentation. (s/f). Python Documentation. Recuperado el 27 de mayo de 2026, de https://docs.python.org/3/ - Python, R. (2020, enero 6). Using pandas and Python to Explore Your Dataset. - Realpython.com; Real Python. https://realpython.com/pandas-python-explore-dataset/



[Handwritten signatures and marks on the right margin]